

INDEX

Sr. No	Title of the Practicals	Date	Pg. No	Sign
1	Basics of Image Processing a. Image Representation b. Sampling (Spatial resolution) & Quantization (Intensity resolution)..	20/02/26	1-8	
2	Mathematical tools for Image Processing a. Arithmetic operations on digital images b. Discrete Cosine Transformation.	20/02/26	9-17	
3	Image Enhancement in Spatial Domain. a. Basic Intensity Transformation on images. b. Histogram Processing. c. Spatial Filtering- Smoothing (LPF) & Sharpening (HPF) Filters.	20/02/26	18-32	
4	Image Enhancement in Frequency Domain. a. Discrete Fourier transformation (DFT) & Inverse DiscreteFourier Transformation (IDFT). b. Low Pass Filters- Ideal, Gaussian & Butterworth. c. High Pass Filters- Ideal, Gaussian & Butterworth.	20/02/26	33-44	
5	Applications of Image Denoising Techniques. a. Image Deblurring Using Inverse Filters. b. Image Deblurring Using Wiener Filters.	20/02/26	45-47	
6	Illustrations of various Segmentation Techniques. a. Edge Detection using sobel, prewitt,canny & Laplacian Operators.		48-52	
7	Applications of Morphological operations on Images. a. Apply Erosion, Dilation, Opening and Closing operations on images.	20/02/26	53-56	

Practical 1

Basics of Image Processing

Aim: To understand and implement basic image processing techniques such as array-based image representation, image manipulation, RGB-HSV conversion, and grayscale transformation using Python.

Software Used: Jupyter Notebook

Theory:

Digital image processing deals with the manipulation and analysis of images represented in matrix form. An image consists of pixels, where each pixel stores intensity or color information. Image manipulation involves operations such as slicing the image matrix, separating color components, color space conversion, and resolution reduction.

Slicing divides the image matrix into smaller sections for region-based analysis. In the RGB model, a color image is represented using Red, Green, and Blue components, each contributing to the final image. Conversion from RGB to HSV separates color information from brightness, making image analysis easier. Grayscale conversion reduces a color image to intensity values, simplifying processing and reducing computational complexity.

Quantization reduction decreases the number of intensity levels, while sampling reduction lowers the spatial resolution of the image. Reducing both quantization and sampling leads to loss of image quality but improves storage efficiency and processing speed. These techniques demonstrate the trade-off between image quality and data reduction in digital image processing.

Implementation:

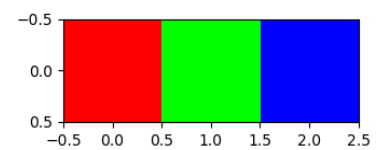
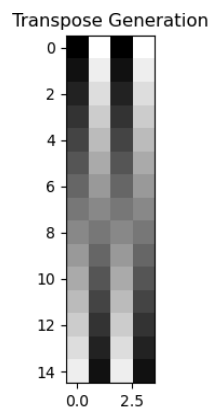
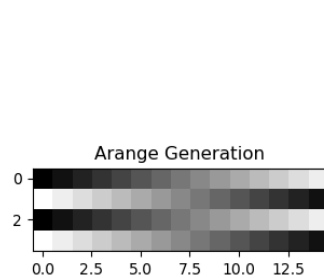
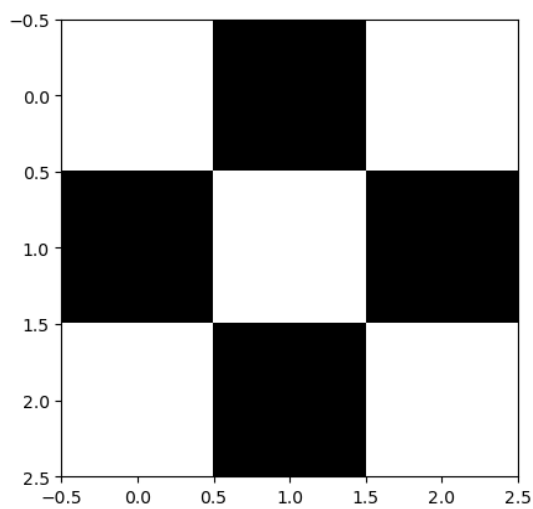
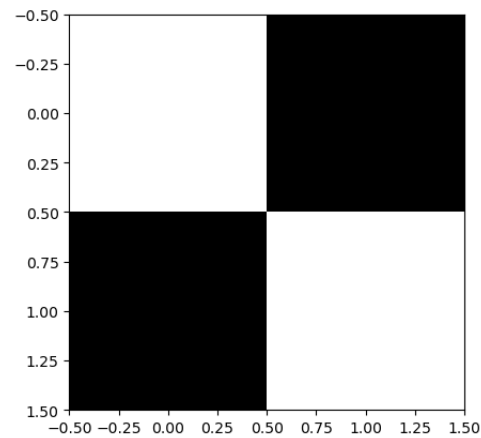
```
import numpy as np
import matplotlib.pyplot as plt
from skimage import img_as_uint
from skimage.io import imshow, imread
from skimage.color import rgb2hsv
from skimage.color import rgb2gray

array_1 = np.array([[255, 0], [0, 255]])
plt.imshow(array_1, cmap = 'gray')
plt.show()

array_2 = np.array([[255, 0, 255], [0, 255, 0], [255, 0, 255]])
plt.imshow(array_2, cmap = 'gray');
plt.show()

array_spectrum = np.array([np.arange(0,255,17),
np.arange(255,0,-17),np.arange(0,255,17),np.arange(255,0,-17)])
fig, ax = plt.subplots(1, 3, figsize=(12,4))
ax[0].imshow(array_spectrum, cmap = 'gray')
ax[0].set_title('Arange Generation')
ax[1].imshow(array_spectrum.T, cmap = 'gray')
ax[1].set_title('Transpose Generation');

array_colors = np.array([[[255, 0, 0], [0, 255, 0],[0, 0, 255]]])
plt.imshow(array_colors);
plt.show()
```

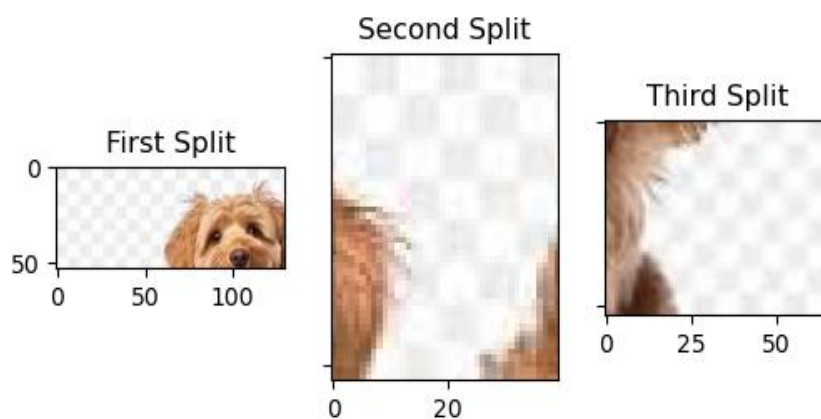
Output:**Manipulating an actual image**

```
import matplotlib.pyplot as plt
import matplotlib
from skimage import img_as_uint
from skimage.io import imshow, imread
from skimage.color import rgb2hsv
from skimage.color import rgb2gray
doggo = imread(r"C:\Users\ADMIN\Desktop\doggo.jpg")
plt.imshow(doggo)
plt.show()
```



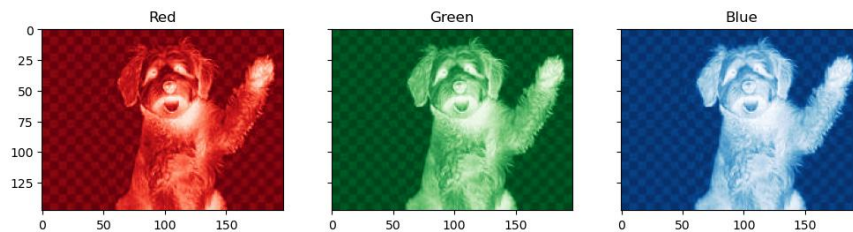
Slice the image matrix and represent each section as its own image box.

```
import matplotlib.pyplot as plt
import matplotlib
from skimage import img_as_uint
from skimage.io import imread
from skimage.color import rgb2hsv
from skimage.color import rgb2gray
doggo = imread(r"C:\Users\ADMIN\Desktop\doggo.jpg")
fig, ax = plt.subplots(1, 3, figsize=(6,4), sharey=True)
ax[0].imshow(doggo[:, 0:130])
ax[0].set_title('First Split')
ax[1].imshow(doggo[:, 130:170])
ax[1].set_title('Second Split')
ax[2].imshow(doggo[:, 170:210])
ax[2].set_title('Third Split');
plt.imshow(doggo[95:250, 130:275])
plt.show()
```



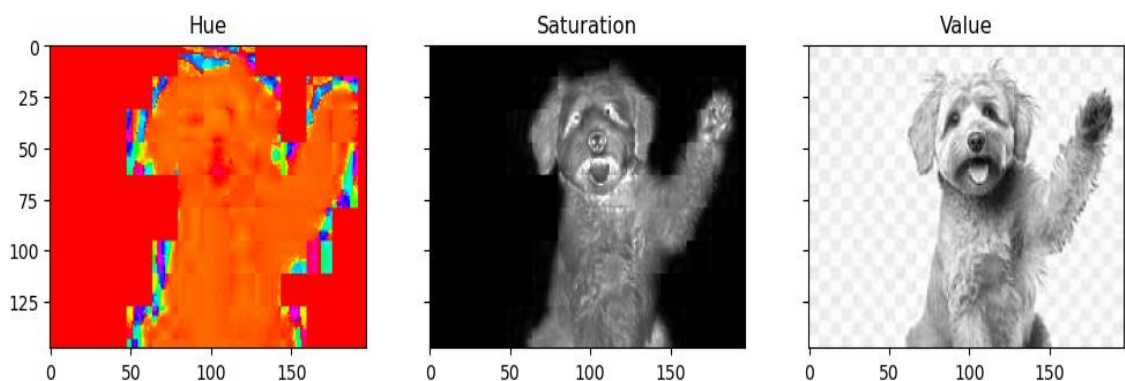
Representation of the images Red, Green, and Blue components

```
import matplotlib.pyplot as plt
import matplotlib
from skimage import img_as_uint
from skimage.io import imshow, imread
from skimage.color import rgb2hsv
from skimage.color import rgb2gray
doggo = imread(r"C:\Users\ADMIN\Desktop\doggo.jpg")
fig, ax = plt.subplots(1, 3, figsize=(12,4), sharey = True)
ax[0].imshow(doggo[:, :, 0], cmap='Reds')
ax[0].set_title('Red')
ax[1].imshow(doggo[:, :, 1], cmap='Greens')
ax[1].set_title('Green')
ax[2].imshow(doggo[:, :, 2], cmap='Blues')
ax[2].set_title('Blue');
```



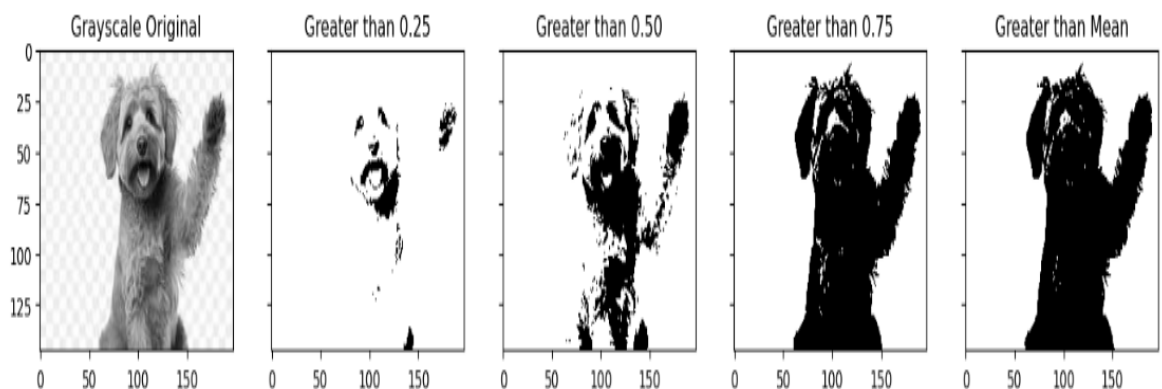
Convert the image from RGB (Red, Green, Blue) to HSV (Hue, Saturation, Value)

```
import matplotlib.pyplot as plt
import matplotlib
from skimage import img_as_uint
from skimage.io import imshow, imread
from skimage.color import rgb2hsv
from skimage.color import rgb2gray
doggo = imread(r"C:\Users\ADMIN\Desktop\doggo.jpg")
doggo_hsv = rgb2hsv(doggo)
fig, ax = plt.subplots(1, 3, figsize=(12,4), sharey = True)
ax[0].imshow(doggo_hsv[:, :, 0], cmap='hsv')
ax[0].set_title('Hue')
ax[1].imshow(doggo_hsv[:, :, 1], cmap='gray')
ax[1].set_title('Saturation')
ax[2].imshow(doggo_hsv[:, :, 2], cmap='gray')
ax[2].set_title('Value');
```



Convert the image matrix to grayscale

```
import matplotlib.pyplot as plt
import matplotlib
from skimage import img_as_uint
from skimage.io import imshow, imread
from skimage.color import rgb2hsv
from skimage.color import rgb2gray
doggo = imread(r"C:\Users\ADMIN\Desktop\doggo.jpg")
doggo_gray = rgb2gray(doggo)
fig, ax = plt.subplots(1, 5, figsize=(17,6), sharey = True)
ax[0].imshow(doggo_gray, cmap = 'gray')
ax[0].set_title('Grayscale Original')
ax[1].imshow(img_as_uint(doggo_gray > 0.25), cmap = 'gray')
ax[1].set_title('Greater than 0.25') |
ax[2].imshow(img_as_uint(doggo_gray > 0.50), cmap = 'gray')
ax[2].set_title('Greater than 0.50');
ax[3].imshow(img_as_uint(doggo_gray > 0.75), cmap = 'gray')
ax[3].set_title('Greater than 0.75');
ax[4].imshow(img_as_uint(doggo_gray > np.mean(doggo_gray)), cmap = 'gray')
ax[4].set_title('Greater than Mean');
```



• Conclusion:

Basic image processing techniques such as slicing, separating color components, converting RGB to HSV, and transforming images to grayscale were carried out effectively. These operations provided a clearer understanding of how images are stored as pixel matrices and how different color models make analysis simpler. Overall, the practical work enhanced the ability to process and manipulate digital images more efficiently.

Practical No:1

Basics of Image Processing

1.(b) Aim: To examine the processes of sampling and quantization, which are essential for digitizing images and determining their quality and detail.

Software used: Google Colab

Theory:

1. Quantization Reduction:

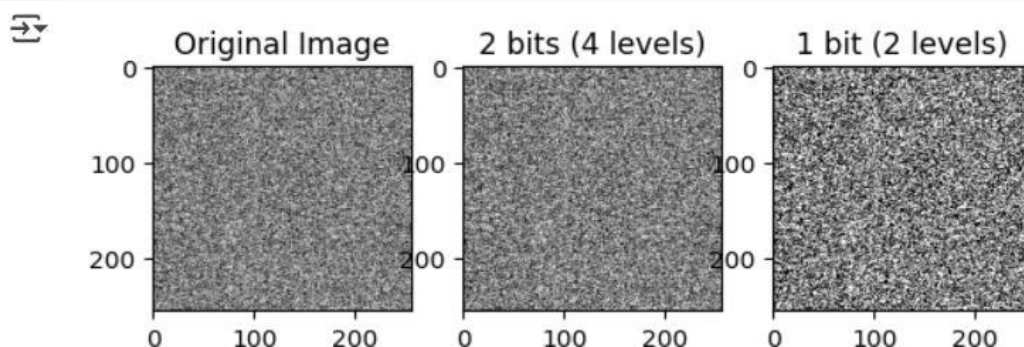
Reduces the number of color/intensity levels in an image

2-bit quantization (4 levels) created by: $\text{np.round}(\text{img} * 3) / 3$

1-bit quantization (2 levels, black/white) created by: $\text{np.round}(\text{img})$

Shows how reducing bit depth increases visible banding/contours

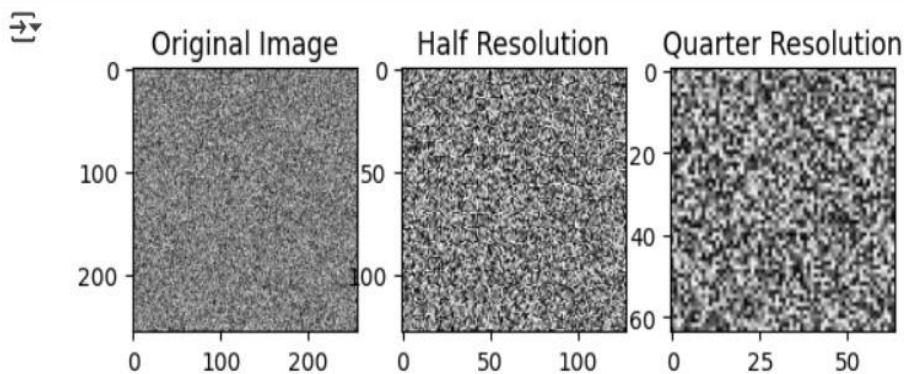
```
import numpy as np
import matplotlib.pyplot as plt
# Create a sample image
img = np.random.rand(256, 256)
# Reduce the quantization values to 2 bits (4 levels)
reduced_img_2bits = np.round(img * 3) / 3
# Reduce the quantization values to 1 bit (2 levels)
reduced_img_1bit = np.round(img)
# Display the images
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.subplot(1, 3, 2)
plt.imshow(reduced_img_2bits, cmap='gray')
plt.title('2 bits (4 levels)')
plt.subplot(1, 3, 3)
plt.imshow(reduced_img_1bit, cmap='gray')
plt.title('1 bit (2 levels)')
plt.show()
```



2. Resolution Reduction (Subsampling)

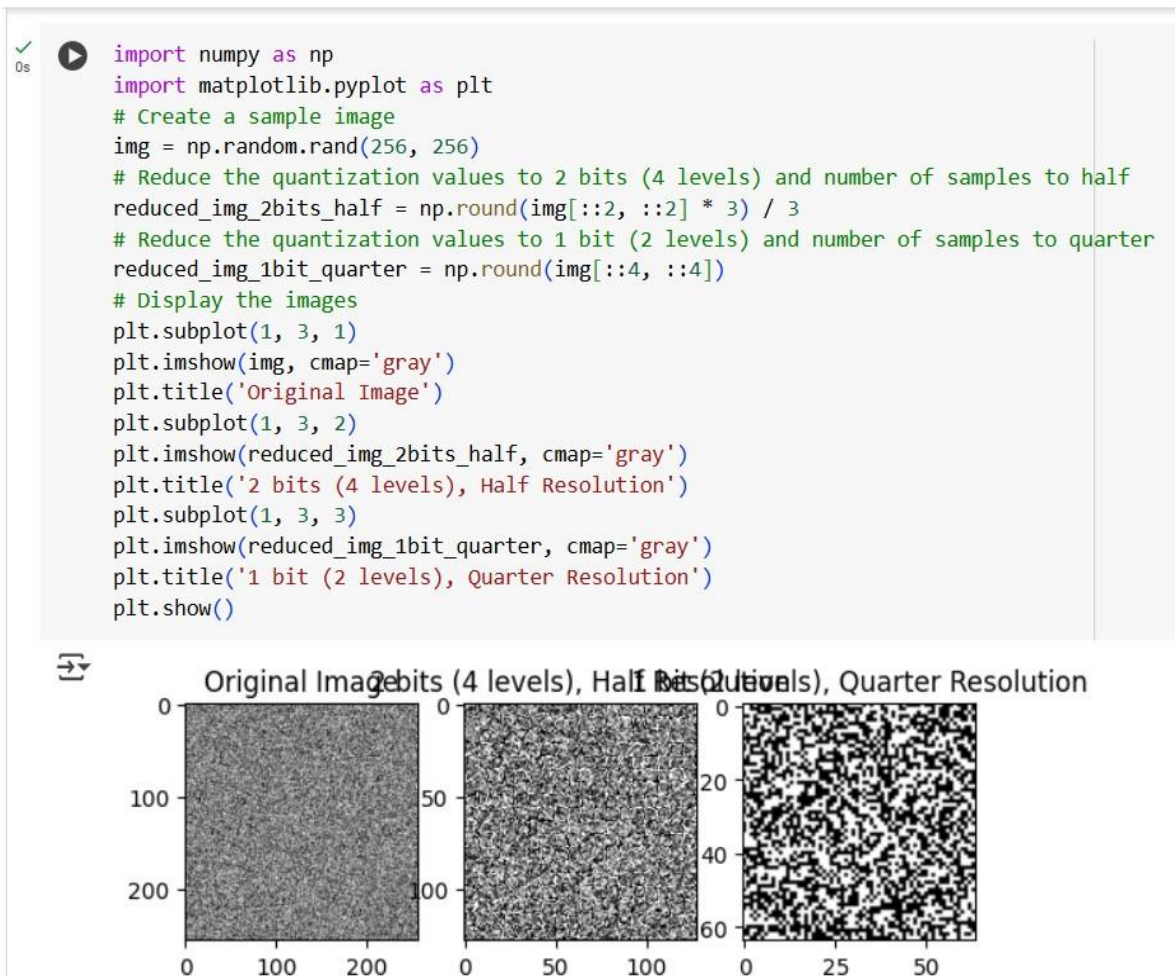
- Reduces pixel count by skipping rows/columns
- Half resolution: `img[::2, ::2]` (takes every 2nd pixel)
- Quarter resolution: `img[::4, ::4]` (takes every 4th pixel)
- Demonstrates how lower resolution makes images blocky/pixelated

```
import numpy as np
import matplotlib.pyplot as plt
# Create a sample image
img = np.random.rand(256, 256)
# Reduce the number of samples to half
reduced_img_half = img[::2, ::2]
# Reduce the number of samples to quarter
reduced_img_quarter = img[::4, ::4]
# Display the images
plt.subplot(1, 3, 1)
plt.imshow(img, cmap='gray')
plt.title('Original Image')
plt.subplot(1, 3, 2)
plt.imshow(reduced_img_half, cmap='gray')
plt.title('Half Resolution')
plt.subplot(1, 3, 3)
plt.imshow(reduced_img_quarter, cmap='gray')
plt.title('Quarter Resolution')
plt.show()
```



3. Combined Effects

- Applies both quantization and resolution reduction together
- Shows compounded degradation when both techniques are used



• Conclusion:

This experiment clearly showed how reducing color depth (quantization) and resolution (subsampling) degrades image quality. Lower bit depths created visible banding, while resolution reduction caused pixelation. Together, these techniques demonstrate the essential trade-off between image quality and file size that underpins digital compression. The results help explain how real-world formats like JPEG balance visual fidelity with storage efficiency.

Practical No:2

Mathematical tools for Image Processing

2.(a) Aim: To utilize basic arithmetic operations such as addition, subtraction, multiplication, and division to analyze and enhance image properties.

Software used: Google Colab

Theory:

1. Image Loading and Conversion:

The practical begins with loading sample images from the skimage library and converting them to OpenCV's preferred color format:

- data.coins() loads a grayscale coin image
- data.astronaut() loads a color astronaut image
- cv2.cvtColor(): Converts between color spaces (though there's a typo in the code - should be COLOR_GRAY2BGR and COLOR_RGB2BGR)
 - Grayscale to BGR adds 3 identical channels to make it compatible with color operations
 - RGB to BGR swaps red and blue channels (OpenCV uses BGR by default)



```
[1] !pip install opencv-python
from IPython import get_ipython
from IPython.display import display

Requirement already satisfied: opencv-python in /usr/local/lib/python3.11/dist-packages (4.11.0.86)
Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.11/dist-packages (from opencv-python) (2.0.2)

[2] import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab.patches import cv2_imshow
from skimage import data

[3] coins_image = data.coins()

[4] coins_image_cv2 = cv2.cvtColor(coins_image, cv2.COLOR_GRAY2BGR)

[5] astronaut_image = data.astronaut()

[6] astronaut_image_cv2 = cv2.cvtColor(astronaut_image, cv2.COLOR_RGB2BGR)

[7] cv2_imshow(coins_image_cv2)
cv2_imshow(astronaut_image_cv2)
```



2. Image Saving and Reloading

The images are saved and reloaded to demonstrate file I/O operations:

- `cv2.imwrite()` saves images to disk
- `cv2.imread()` reads images from disk

```
✓ [8] cv2.imwrite('coins_image_cv2.jpg',coins_image_cv2)
js    cv2.imwrite('astronaut_image_cv2.jpg',astronaut_image_cv2)
      image1 = cv2.imread('coins_image_cv2.jpg')
      image2 = cv2.imread('astronaut_image_cv2.jpg')
```

```
✓ [9] print(f"Original Image 1 shape: {image1.shape}")
js    print(f"Original Image 2 shape: {image2.shape}")
```

```
⇒ Original Image 1 shape: (303, 384, 3)
   Original Image 2 shape: (512, 512, 3)
```

3. Image Resizing

The coins image is resized to match the astronaut image dimensions:

- `cv2.resize()` performs interpolation to scale the image
- Necessary for arithmetic operations which require same-sized images

```
image1_resized = cv2.resize(image1, (image2.shape[1], image2.shape[0]))
```

4. Image Arithmetic Operations

Several fundamental arithmetic operations are demonstrated:

a. Image Addition (`cv2.add()`)

- Pixel values are added (capped at 255)
- Results in a brighter composite image combining features from both

b. Image Subtraction (`cv2.subtract()`)

- Pixel values are subtracted (minimum 0)
- Highlights differences between images

c. Image Multiplication (`cv2.multiply()`)

- Each pixel is multiplied by a scalar (0.5 in this case)
- Reduces brightness (darkens the image)

d. Image Division (`cv2.divide()`)

- Each pixel is divided by a scalar (2 in this case)
- Similar effect to multiplication but with different numerical properties

```

✓ [11] added_image = cv2.add(image1_resized, image2)
0s subtracted_image = cv2.subtract(image1_resized, image2)
multiplied_image = cv2.multiply(image1_resized, 0.5)
divided_image = cv2.divide(image1_resized, 2)

✓ 2s ▶ fig, axes = plt.subplots(3, 3, figsize=(15, 15))
# Original Images
axes[0, 0].imshow(cv2.cvtColor(image1, cv2.COLOR_BGR2RGB))
axes[0, 0].set_title(f'Original Image 1 ({image1.shape[1]}x{image1.shape[0]})')
axes[0, 0].axis('off')

axes[0, 1].imshow(cv2.cvtColor(image2, cv2.COLOR_BGR2RGB))
axes[0, 1].set_title(f'Original Image 2 ({image2.shape[1]}x{image2.shape[0]})')
axes[0, 1].axis('off')

# Resized Image 1
axes[0, 2].imshow(cv2.cvtColor(image1_resized, cv2.COLOR_BGR2RGB))
axes[0, 2].set_title(f'Resized Image 1 ({image1_resized.shape[1]}x{image1_resized.shape[0]})')
axes[0, 2].axis('off')

# Operation Results
axes[1, 0].imshow(cv2.cvtColor(added_image, cv2.COLOR_BGR2RGB))
axes[1, 0].set_title(f'Added Image ({added_image.shape[1]}x{added_image.shape[0]})')
axes[1, 0].axis('off')

axes[1, 1].imshow(cv2.cvtColor(subtracted_image, cv2.COLOR_BGR2RGB))
axes[1, 1].set_title(f'Subtracted Image ({subtracted_image.shape[1]}x{subtracted_image.shape[0]})')
axes[1, 1].axis('off')

▶ axes[1, 2].axis('off') # Empty subplot for better layout

axes[2, 0].imshow(cv2.cvtColor(multiplied_image, cv2.COLOR_BGR2RGB))
axes[2, 0].set_title(f'Multiplied Image (x0.5) ({multiplied_image.shape[1]}x{multiplied_image.shape[0]})')
axes[2, 0].axis('off')

axes[2, 1].imshow(cv2.cvtColor(divided_image, cv2.COLOR_BGR2RGB))
axes[2, 1].set_title(f'Divided Image (/2) ({divided_image.shape[1]}x{divided_image.shape[0]})')
axes[2, 1].axis('off')

axes[2, 2].axis('off') # Empty subplot for better layout

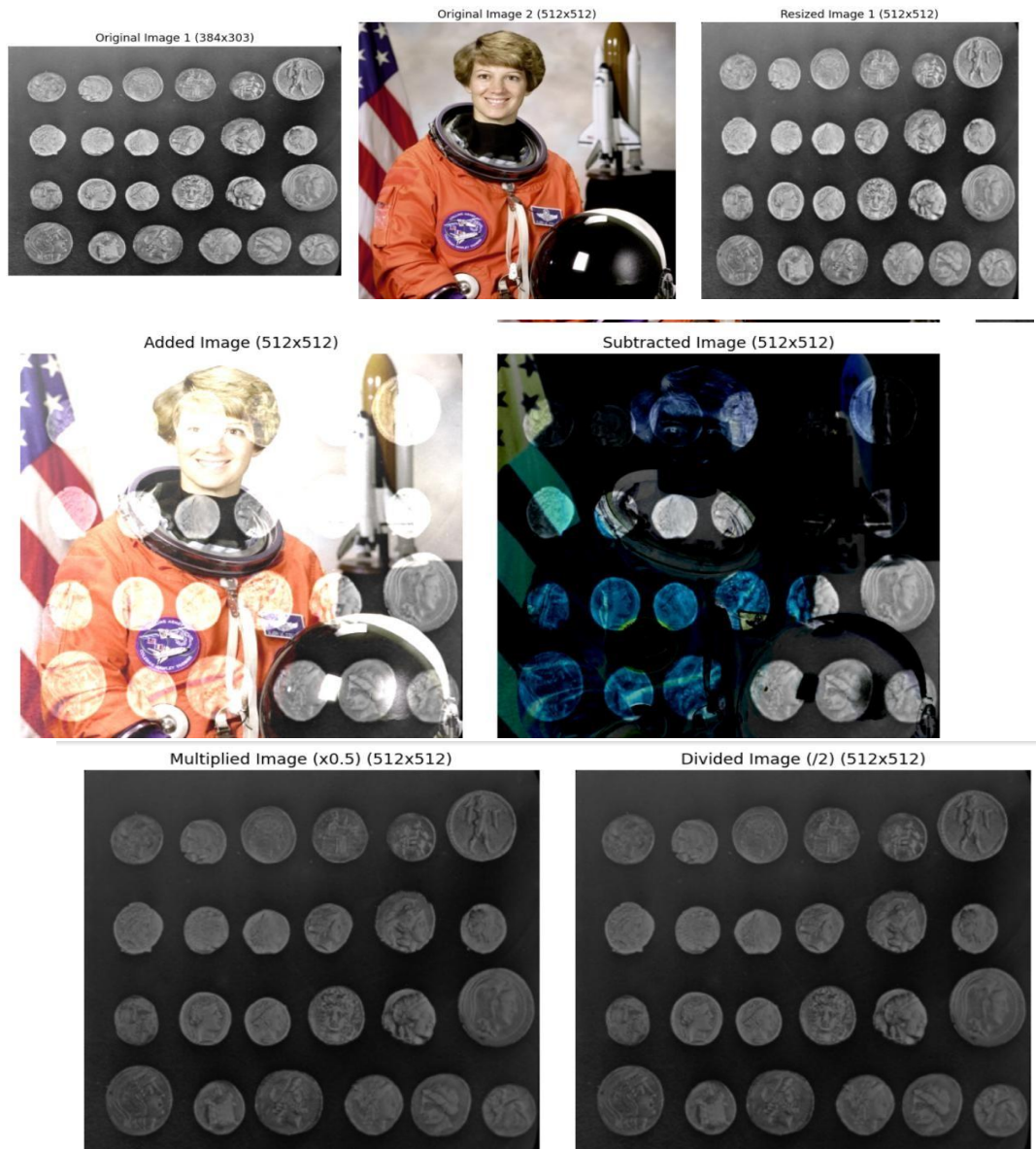
plt.tight_layout()
plt.show()

```

5. Visualization

The results are displayed using matplotlib:

- `plt.subplots()` creates a grid of subplots
- `imshow()` displays images with proper color conversion (`COLOR_BGR2RGB`)
- Titles show image dimensions and operation performed



- **Conclusion:**

In conclusion, this practical demonstrated essential image processing techniques using OpenCV, including color conversion, resizing, and arithmetic operations like addition, subtraction, multiplication, and division. The results showed how these operations manipulate pixel values to blend images, adjust brightness, and highlight differences. The exercise confirmed that these fundamental techniques form the basis for more advanced computer vision tasks, while emphasizing the importance of proper color management and dimension matching. The clear visualization of results reinforced key concepts, providing valuable hands-on experience with core image processing workflows.

Practical No:2**Mathematical tools for Image Processing**

2.(b) Aim: To apply the Discrete Cosine Transformation for converting image data to the frequency domain, enabling efficient compression and representation.

Software used: Google Colab

Theory:**1. Discrete Cosine Transform (DCT):**

The Discrete Cosine Transform (DCT) is a widely used signal processing technique that converts spatial-domain image data into frequency-domain coefficients. It is particularly useful in image compression (e.g., JPEG) and feature extraction.

Key Properties of DCT:

Separable Transform: Can be applied along rows and columns independently.

Energy Compaction: Most significant coefficients are concentrated in the topleft corner (low-frequency components).

Lossless Reconstruction: Applying Inverse DCT (IDCT) perfectly reconstructs the original image if no coefficients are discarded.

Mathematical Formulation (2D-DCT):

$$C(u, v) = \alpha(u)\alpha(v) \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos\left(\frac{(2x+1)u\pi}{2N}\right) \cos\left(\frac{(2y+1)v\pi}{2N}\right)$$

where:

- $\alpha(u) = \begin{cases} \frac{1}{\sqrt{N}} & \text{if } u = 0 \\ \sqrt{\frac{2}{N}} & \text{otherwise} \end{cases}$
- $f(x, y)$ = pixel intensity at position (x, y)
- $C(u, v)$ = DCT coefficient at frequency (u, v)

```
[1] !pip install opencv-python
    from IPython import get_ipython
    from IPython.display import display
```

Requirement already satisfied: opencv-python in /usr/local/lib/python3.11/dist-packages (4.11.0.86)
Requirement already satisfied: numpy>=1.21.2 in /usr/local/lib/python3.11/dist-packages (from opencv-python) (2.0.2)

```
[2] import cv2
    import numpy as np
    import matplotlib.pyplot as plt
    from google.colab.patches import cv2_imshow
    from skimage import data
```

```
[3] coins_image = data.coins()
    coins_image_cv2 = coins_image
```

```
dct_image = cv2.dct(np.float32(coins_image_cv2))
idct_image = cv2.idct(dct_image)
plt.figure(figsize=(12, 6))
plt.subplot(1, 3, 1)
plt.title('Original Image')
plt.imshow((coins_image_cv2), cmap='gray') # Original image is grayscale
plt.axis('off')
plt.subplot(1, 3, 2)
plt.title('DCT Image')
plt.imshow(np.log(abs(dct_image) + 1), cmap='gray') # Log scale for better visibility
plt.axis('off')
plt.subplot(1, 3, 3)
plt.title('Reconstructed Image')
plt.imshow(idct_image, cmap='gray') # Reconstructed image is grayscale
plt.axis('off')
plt.show()
```



Original Image



DCT Image



Reconstructed Image



a. Loading and Preprocessing the Image

- **cv2.imread()**: Reads an image (e.g., lena.jpg in grayscale).
- **astype(np.float64)**: Converts pixel values to floating-point for numerical stability in DCT computation.

```
[5] !pip install opencv-python
    !pip install matplotlib
    !pip install scikit-image
```

```
✓ [6] import cv2
0s      import numpy as np
      import matplotlib.pyplot as plt
```

```
✓ # Load lena.jpg image
0s im = cv2.imread('/content/lena.jpg', cv2.IMREAD_GRAYSCALE) # Read as grayscale
    im = im.astype(np.float64) # Convert to double precision (like MATLAB's double)
```

```
✓ plt.figure(figsize=(12, 4))
0s plt.subplot(1, 3, 1)
    plt.imshow(im, cmap='gray')
    plt.title('Original')
    plt.axis('off')
```

```
✓ [8] plt.figure(figsize=(12, 4))
0s plt.subplot(1, 3, 1)
    plt.imshow(im, cmap='gray')
    plt.title('Original')
    plt.axis('off')
```

```
⇒ (np.float64(-0.5), np.float64(319.5), np.float64(319.5), np.float64(-0.5))
```

Original



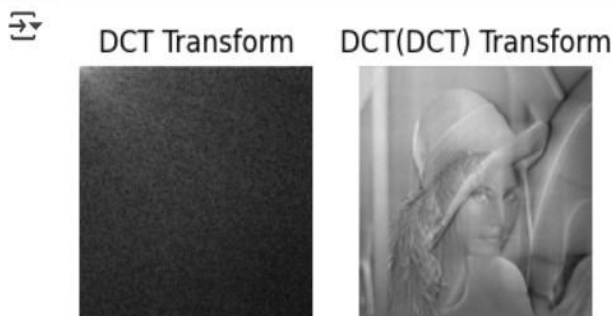
b. Applying DCT

- **cv2.dct()**: Computes the **forward DCT** of the image.
 - Input: Floating-point grayscale image.
 - Output: Frequency-domain coefficients (top-left = low frequencies, bottom-right = high frequencies).
- **Log Scaling (np.log(abs(dct_image) + 1))**: Enhances visibility of lowmagnitude coefficients.

c. Visualizing Results

- **plt.subplot():** Displays the original, DCT, and reconstructed images side-by-side.
- **cmap='gray':** Ensures grayscale visualization.

```
# Apply DCT (Discrete Cosine Transform) - Using OpenCV
dct_image = cv2.dct(np.float32(im))
plt.subplot(1, 3, 2)
plt.imshow(np.log(abs(dct_image) + 1), cmap='gray') # Log scale for better visibility
plt.title('DCT Transform')
plt.axis('off')
# Apply DCT again to the DCT result
dct_dct_image = cv2.dct(np.float32(dct_image))
plt.subplot(1, 3, 3)
plt.imshow(dct_dct_image, cmap='gray') # You might need to adjust scaling here
plt.title('DCT(DCT) Transform')
plt.axis('off')
plt.show()
```



- **Conclusion:**

This practical demonstrated the **Discrete Cosine Transform (DCT)** using OpenCV. Key takeaways include:

- **DCT efficiently represents images in the frequency domain**, with most information concentrated in low-frequency coefficients.
- **Lossless reconstruction** is possible with IDCT, making DCT useful in compression (e.g., JPEG).
- **Log scaling** helps visualize weak high-frequency components.
- **Double DCT** is mathematically valid but rarely used in practice since it does not improve compression.

Practical No:3

Image Enhancement in Spatial Domain.

3.(a) Aim: To implement intensity transformation techniques for adjusting brightness, contrast, and other visual attributes to improve image quality.

Software used: Google Colab

Theory:

1. Basic Intensity Transformations

Intensity transformations modify pixel values to enhance or extract specific image features. The practical demonstrates several fundamental techniques:

Negative Transformation:

- Concept: Inverts pixel intensities ($I' = L-1-I$ where $L=256$ for 8-bit images)
- Purpose: Enhances dark regions in medical/satellite images
- Implementation: `negative_image = 255 - original_image`

Basic Intensity Transformations on Images using Python

1.Linear (negative transformations)

```
✓ 1s ▶ import cv2
import matplotlib.pyplot as plt
# Load the original image
original_image = cv2.imread("lena.jpg") # Replace with your image file
# Check if the image was loaded successfully
# Perform the negative transformation
negative_image = 255 - original_image
# Display the original and negative images side by side
plt.figure(figsize=(10, 5)) # Create a figure with a specified size
plt.subplot(1, 2, 1) # Subplot for the original image
plt.imshow(cv2.cvtColor(original_image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis('off') # Turn off axis labels
plt.subplot(1, 2, 2) # Subplot for the negative image
plt.imshow(cv2.cvtColor(negative_image, cv2.COLOR_BGR2RGB))
plt.title("Negative Image")
plt.axis('off')
plt.show() # Show the figure with both images
```




Logarithmic Transformation

- Concept: Compresses dynamic range ($s = c \cdot \log(1+r)$)
- Purpose: Reveals details in dark areas
- Implementation: $\text{log_transformed} = c * \text{np.log}(\text{image} + 1)$

2. Log Transformation (Logarithm Function)&Inverse-Log Transformation (Exponential Function)

```

import cv2
import numpy as np
# Load an image
image = cv2.imread('owl.jpeg', cv2.IMREAD_GRAYSCALE)
# Log Transformation
c = 45
log_transformed = c * (np.log(image + 1))
# Convert to 8-bit unsigned integer format
log_transformed = np.uint8(log_transformed)
# Inverse-Log Transformation
inverse_log_transformed = c * (np.exp(log_transformed / c) - 1)
# Convert to 8-bit unsigned integer format
inverse_log_transformed = np.uint8(inverse_log_transformed)
# Display the original and negative images side by side
plt.figure(figsize=(10, 5)) # Create a figure with a specified size
plt.subplot(1, 3, 1) # Subplot for the original image
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis('off') # Turn off axis labels
plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(log_transformed, cv2.COLOR_BGR2RGB))
plt.title("log_transformed")
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(inverse_log_transformed, cv2.COLOR_BGR2RGB))
plt.title("inverse_log_transformed")
plt.axis('off')
plt.show()

```

```

<ipython-input-2-bc18b4e9b410>:7: RuntimeWarning: divide by zero encountered in log
log_transformed = c * (np.log(image + 1))
<ipython-input-2-bc18b4e9b410>:9: RuntimeWarning: invalid value encountered in cast
log_transformed = np.uint8(log_transformed)

```

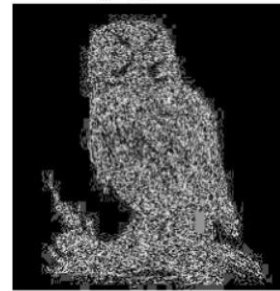
Original Image



log_transformed



inverse_log_transformed



Power-Law (Gamma) Transformation

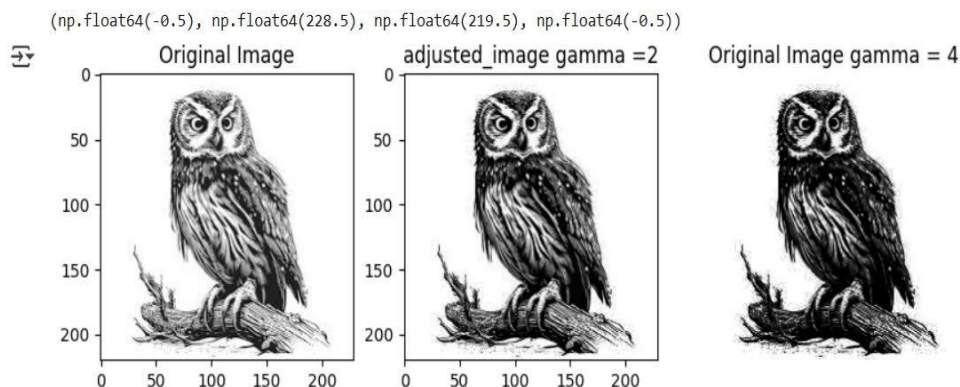
- Concept: Non-linear mapping ($s = cr^\gamma$)
- Purpose: Controls contrast ($\gamma < 1$ enhances dark tones, $\gamma > 1$ enhances bright tones)
- Implementation: `adjusted = np.power(image/255.0, gamma) * 255.0`

POWER LAW TRANSFORMATIONS

```

import cv2
import numpy as np
# Read an image
image = cv2.imread('owl.jpeg', cv2.IMREAD_GRAYSCALE)
# Apply gamma correction (e.g., gamma = 1.5)
gamma = 2
gamma2=4
adjusted_image = np.power(image / 255.0, gamma) * 255.0
adjusted_image = adjusted_image.astype(np.uint8)
adjusted_image2 = np.power(image / 255.0, gamma2) * 255.0
adjusted_image2 = adjusted_image2.astype(np.uint8)
plt.figure(figsize=(10, 5)) # Create a figure with a specified size
plt.subplot(1, 3, 2) # Subplot for the original image
plt.imshow(cv2.cvtColor(adjusted_image, cv2.COLOR_BGR2RGB))
plt.title("adjusted_image gamma =2")
plt.subplot(1, 3, 1) # Subplot for the original image
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.subplot(1, 3, 3) # Subplot for the original image
plt.imshow(cv2.cvtColor(adjusted_image2, cv2.COLOR_BGR2RGB))
plt.title("Original Image gamma = 4")
plt.axis('off')

```



2. Advanced Intensity Operations

Contrast Stretching

- Concept: Piecewise linear transformation to expand intensity range
- Purpose: Improves low-contrast images
- Implementation: Custom pixelVal() function with three linear segments

contrast stretching

```

import cv2
import numpy as np
# Function to map each intensity level to output intensity level.
def pixelVal(pix, r1, s1, r2, s2):
    if (0 <= pix and pix <= r1):
        return (s1 / r1)*pix
    elif (r1 < pix and pix <= r2):
        return ((s2 - s1)/(r2 - r1)) * (pix - r1) + s1
    else:
        return ((255 - s2)/(255 - r2)) * (pix - r2) + s2
# Open the image.
img = cv2.imread('/content/horse.jpeg')
# Define parameters.
r1 = 70
s1 = 0
r2 = 140
s2 = 255
# Vectorize the function to apply it to each value in the Numpy array.
pixelVal_vec = np.vectorize(pixelVal)
# Apply contrast stretching.
contrast_stretched = pixelVal_vec(img, r1, s1, r2, s2)
# Save edited image.
cv2.imwrite('contrast_stretch.jpg', contrast_stretched)
# Display the original and contrast-stretched images side by side
plt.figure(figsize=(10, 5)) # Create a figure with a specified size
plt.subplot(1, 2, 1) # Subplot for the original image
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")

plt.subplot(1, 2, 2) # Subplot for the contrast-stretched image
plt.imshow(cv2.cvtColor(contrast_stretched.astype(np.uint8),
cv2.COLOR_BGR2RGB))
plt.title('Contrast Stretched Image')
plt.axis('off') # Turn off axis labels
plt.show()

```



Original Image



Contrast Stretched Image



Thresholding

- Concept: Binary classification of pixels
- Purpose: Image segmentation/simplification
- Implementation: cv2.threshold() with THRESH_BINARY mode

Thresholding

```
import cv2
import matplotlib.pyplot as plt
def threshold_image(image_path, threshold_value):
    # Load the image in grayscale
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    # Apply thresholding
    ret, thresholded_img = cv2.threshold(img, threshold_value, 255,
cv2.THRESH_BINARY)
    # Display the original and thresholded images
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(img, cmap='gray')
    plt.title("Original Image")
    plt.axis('off')
    plt.subplot(1, 2, 2)
    plt.imshow(thresholded_img, cmap='gray')
    plt.title("Thresholded Image")
    plt.axis('off')
    plt.show()
# Example usage
threshold_image("lena.jpg", 128)
# Replace with your image path and desired threshold value
```

**Intensity Level Slicing**

- Concept: Highlight specific intensity ranges • Variations:
 - Without background: Replaces non-target pixels with black
 - With background: Preserves original non-target pixels
- Implementation: Vectorized conditional operations

Intensity Level Slicing-

without BG

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image
img = cv2.imread('/content/heart.jpeg')

# Define the intensity level slicing function
def intensityLevelSlicing(pixel_value):
    if 140 < pixel_value < 210:
        return 255 # Set to white
    else:
        return 0 # Set to black

# Apply the function to each pixel in the image
intensityLevelSlicing_vec = np.vectorize(intensityLevelSlicing)
transformed_image = intensityLevelSlicing_vec(img)

# Display the original and transformed images
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 2, 2)

```

```

plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(transformed_image.astype(np.uint8), cv2.COLOR_BGR2RGB))
plt.title("Intensity Level Sliced Image")
plt.axis('off')
plt.show()

```



Original Image



shutterstock.com · 2256785687

Intensity Level Sliced Image



shutterstock.com · 2256785687

with BG

```

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Load the image
img = cv2.imread('/content/heart.jpeg')

# Define the intensity level slicing function with background preservation
def intensityLevelSlicingWithBackground(pixel_value):
    if 140 < pixel_value < 210:
        return 255 # Set to white
    else:
        return pixel_value # Preserve original pixel value

# Apply the function to each pixel in the image
intensityLevelSlicing_vec = np.vectorize(intensityLevelSlicingWithBackground)
transformed_image = intensityLevelSlicing_vec(img)

# Display the original and transformed images
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Original Image")
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(transformed_image.astype(np.uint8), cv2.COLOR_BGR2RGB))
plt.title("Intensity Level Sliced Image with Background")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(transformed_image.astype(np.uint8), cv2.COLOR_BGR2RGB))
plt.title("Intensity Level Sliced Image with Background")
plt.axis('off')

plt.show()

```



Original Image



shutterstock.com · 2256785687

Intensity Level Sliced Image with Background



shutterstock.com · 2256785687

- **Conclusion:**

This practical demonstrated essential image intensity transformations, including negative, logarithmic, power-law, contrast stretching, thresholding, and intensity slicing. Each technique effectively modified pixel values to enhance specific image characteristics, from revealing dark details to isolating intensity ranges. The results clearly showed how simple mathematical operations can improve image quality and extract meaningful information. These fundamental techniques form the basis for more advanced image processing and computer vision applications, proving particularly valuable for medical imaging, surveillance, and segmentation tasks. The hands-on implementation reinforced key concepts while highlighting the importance of parameter selection in achieving desired results.

Practical No:3**Image Enhancement in Spatial Domain.**

3.(b) Aim: To enhance images by modifying their histograms, improving contrast, and analyzing intensity distributions.

Software used: Google Colab

Theory:**1. Bit Plane Slicing**

What the Code Does:

Takes an 8-bit grayscale image (like "lena.jpg")

Splits it into 8 individual binary images (Bit 0 to Bit 7) using:

python

`bit_plane = (img >> i) & 1` # Isolates the i-th bit

Scales each plane to 0 (black) or 255 (white) for visibility

What the Output Shows:

Bit Plane 7 (MSB): Looks like a rough sketch of the original image (most important for structure)

Bit Plane 0 (LSB): Shows random noise (least important visually) •

Middle Planes (Bits 6-1): Gradually reveal finer details

Why It Matters: Proves that image data is stored hierarchically (MSBs > LSBs)

Explains how JPEG compression works (discards noisy LSBs)

```

Bit plane Slicing

3B. Plotting Histogram and histogram Equalization

# Install the google-colab package (only necessary in Colab)
!pip install google-colab

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Define the bit plane slicing function
def bit_plane_slicing(image_path):
    # Read the image in grayscale
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        print("Error: Image not found.")
        return

    num_rows = 2
    num_cols = 4

    # Create a figure and subplots
    fig, axes = plt.subplots(num_rows, num_cols, figsize=(12, 6))
    fig.suptitle("Bit Plane Slicing", fontsize=16)

    # Iterate through each bit plane (0-7)
    for i in range(8):
        bit_plane = (img >> i) & 1
        # Scale the bit plane to 0-255 for visibility
        bit_plane_scaled = bit_plane * 255
        # Plot the bit plane
        axes[i // num_cols, i % num_cols].imshow(bit_plane_scaled, cmap='gray')
        axes[i // num_cols, i % num_cols].axis('off')
    
```

```

# Iterate through each bit plane (0-7)
for i in range(8):
    # Extract the bit plane
    bit_plane = (img >> i) & 1 # Right shift and bitwise AND to get the i-th bit plane

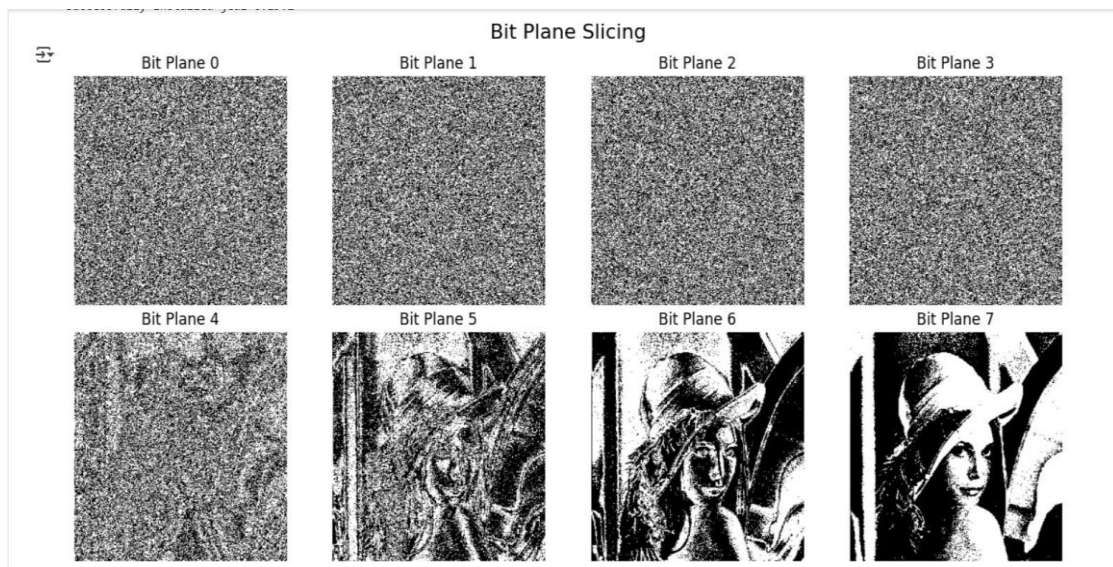
    # Calculate subplot index
    row_index = i // num_cols
    col_index = i % num_cols

    # Display the bit plane in the subplot
    axes[row_index, col_index].imshow(bit_plane * 255, cmap='gray') # Scale for visibility (0 or 255)
    axes[row_index, col_index].set_title(f"Bit Plane {i}")
    axes[row_index, col_index].axis('off') # Turn off axis labels

plt.tight_layout() # Adjust subplot spacing to avoid overlap
plt.show()

# Example usage
bit_plane_slicing("/content/lena.jpg") # Replace with your image path

```



2. Histogram Equalization What the Code Does:

1. Calculates the original histogram (count of pixels at each brightness level)
2. Uses `cv2.equalizeHist()` to automatically:

- Flatten the histogram
- Stretch contrast across the full 0-255 range

What the Output Shows:

- Original Histogram: Peaks indicate overrepresented gray levels
- Equalized Histogram: More evenly distributed bars
- Equalized Image: More visible details in dark/bright areas

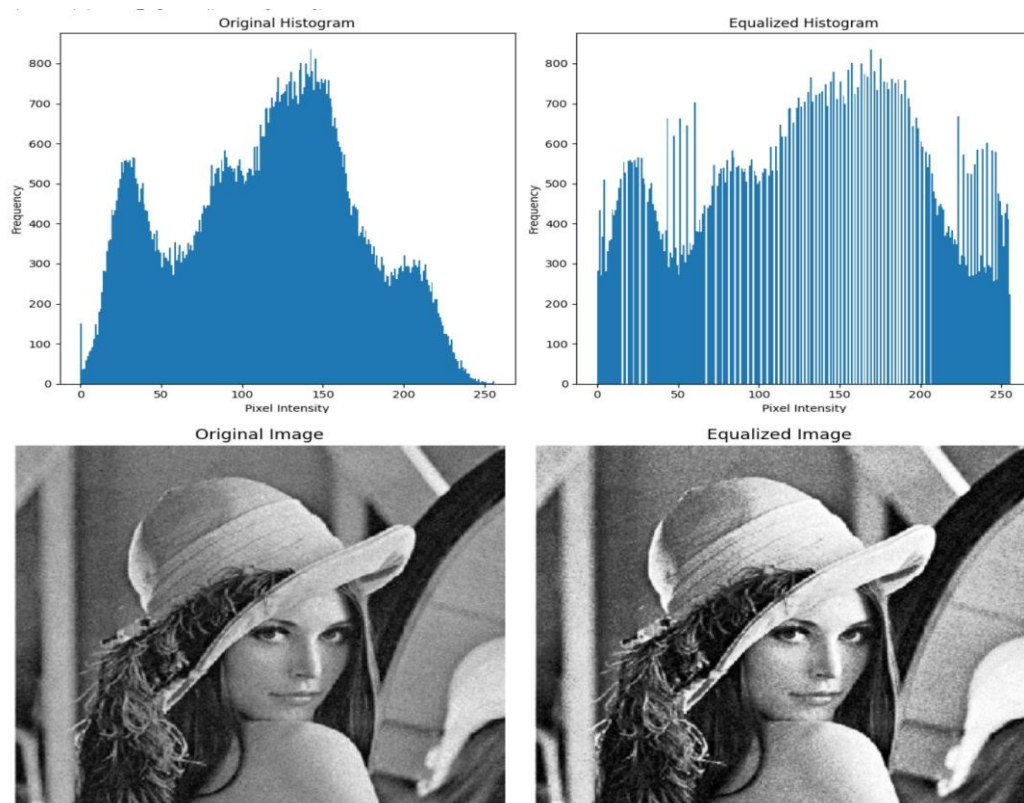
Why It Matters:

- Demonstrates how to fix poorly exposed photos programmatically
- Shows the math behind automatic contrast adjustment in photo editors


```

import cv2
import matplotlib.pyplot as plt
def plot_histogram_and_equalize(image_path):
    """
    Plots the histogram of an image and performs histogram equalization.
    Args:
    image_path: The path to the input image.
    """
    # Load the image in grayscale
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    # Calculate and plot the histogram
    plt.figure(figsize=(12, 6))
    plt.subplot(1, 2, 1)
    plt.hist(img.ravel(), 256, [0, 256])
    plt.title("Original Histogram")
    plt.xlabel("Pixel Intensity")
    plt.ylabel("Frequency")
    # Perform histogram equalization
    equalized_img = cv2.equalizeHist(img)
    # Calculate and plot the histogram of the equalized image
    plt.subplot(1, 2, 2)
    plt.hist(equalized_img.ravel(), 256, [0, 256])
    plt.title("Equalized Histogram")
    plt.xlabel("Pixel Intensity")
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.show()
    # Display the original and equalized images
    plt.figure(figsize=(10, 5))
    plt.subplot(1, 2, 1)
    plt.imshow(img, cmap='gray')
    plt.title("Original Image")
    plt.axis('off')
    plt.subplot(1, 2, 2)
    plt.imshow(equalized_img, cmap='gray')
    plt.title("Equalized Image")
    plt.axis('off')
    plt.tight_layout()
    plt.show()
# Example usage
plot_histogram_and_equalize("/content/lena.jpg") # Replace with your image path

```



- **Conclusion:**

This practical demonstrated two core image processing techniques through hands-on implementation. The bit plane slicing output clearly showed how higher bits (Bit 7-4) contain most visual information while lower bits (Bit 3-0) store finer details and noise. The histogram equalization results proved how redistributing pixel intensities can enhance image contrast, as seen in the more balanced histogram and improved visibility in the equalized image. Both exercises effectively revealed how digital images store and represent visual data at fundamental levels.

Practical No:3

Image Enhancement in Spatial Domain.

3.(c) Aim: To apply filters for reducing noise with smoothing techniques and improving sharpness with edge-enhancing filters.

Software used: Google Colab

Theory:

1. Smoothing Filters (Low-Pass)

These filters reduce noise and blur details by averaging pixel neighborhoods:

Mean Filter:

- Function: cv2.blur()
- Action: Replaces each pixel with average of its neighborhood
- Kernel: Uniform weights (e.g., 5×5 box)
- Effect: Strong noise reduction but causes edge blurring **Gaussian**

Filter:

- Function: cv2.GaussianBlur()
- Action: Weighted average using Gaussian distribution
- Effect: Better noise reduction with smoother transitions **Median**

Filter:

- Function: cv2.medianBlur()
- Action: Replaces pixel with median of neighborhood
- Effect: Excellent for salt-and-pepper noise without blurring edges

3C. Smoothing and Sharpening Filters

Apply and display the results of mean, Gaussian, and median filters on the image

```
✓ [7] import cv2  
0s import numpy as np  
import matplotlib.pyplot as plt
```

```
✓ [8] image = cv2.imread('/content/lena.jpg', cv2.IMREAD_GRAYSCALE)
```

```

✓ 2s # 1. Mean Filter
kernel_size = (5, 5)
mean_filtered = cv2.blur(image, kernel_size)
# 2. Gaussian Filter
gaussian_filtered = cv2.GaussianBlur(image, kernel_size, 0)
# 3. Median Filter
median_kernel_size = 5 # Must be odd
median_filtered = cv2.medianBlur(image, median_kernel_size)
# Display Results
plt.figure(figsize=(15, 5))
plt.subplot(1, 4, 1)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.axis('off')
plt.subplot(1, 4, 2)
plt.imshow(mean_filtered, cmap='gray')
plt.title("Mean Filtered")
plt.axis('off')
plt.subplot(1, 4, 3)
plt.imshow(gaussian_filtered, cmap='gray')
plt.title("Gaussian Filtered")
plt.axis('off')
plt.subplot(1, 4, 4)
plt.imshow(median_filtered, cmap='gray')
plt.title("Median Filtered")
plt.axis('off')
plt.tight_layout()
plt.show()

```



2. Sharpening Filters (High-Pass) These enhance edges and fine details:

Laplacian Filter:

- Function: `cv2.Laplacian()`
- Action: Highlights intensity discontinuities
- Effect: Shows edges as white/dark lines on gray background

Custom High-Pass Kernel:

- Function: `cv2.filter2D()`
- Kernel: Emphasizes center pixel (e.g., `[[-1,-1,-1],[-1,8,-1],[-1,-1,-1]]`)
- Effect: Strong edge enhancement with detail preservation

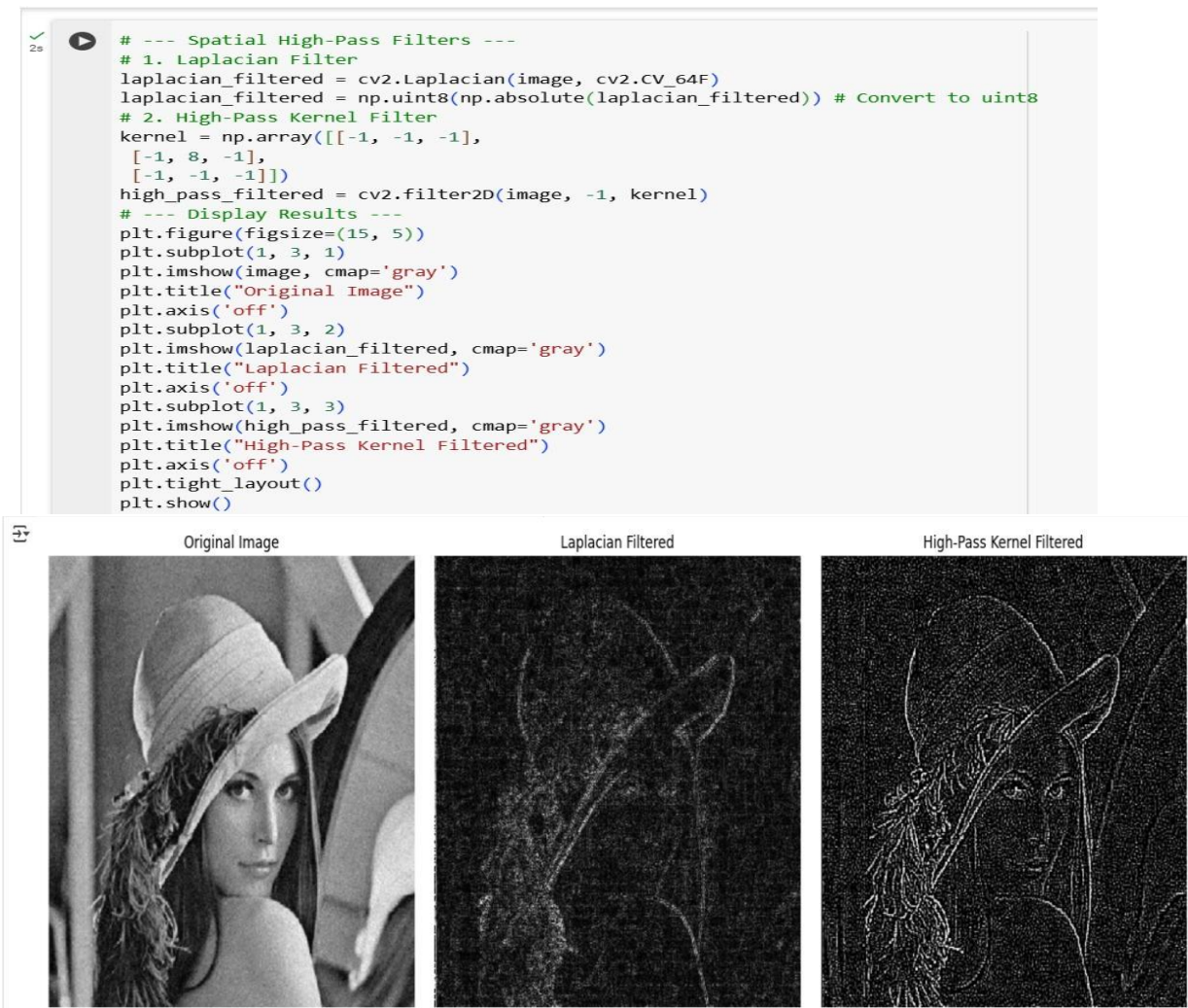
Sharpening (HIGH PASS Filters) Filters: Using the Laplacian operator and using a kernel to emphasize high frequencies.

```

✓ [4] import cv2
import numpy as np
import matplotlib.pyplot as plt

✓ # image = cv2.imread('/content/lena.jpg', cv2.IMREAD_GRAYSCALE)

```



- **Conclusion:**

This practical demonstrated essential spatial filtering techniques for image processing. Smoothing filters effectively reduced noise but required careful balancing to avoid excessive blurring, with the median filter showing particular promise for noise removal. Sharpening filters successfully enhanced edges and details, though they also amplified existing noise. The results highlight the importance of selecting appropriate filter types and parameters based on specific image characteristics and processing goals. These fundamental techniques form the basis for more advanced image enhancement and computer vision applications.

Practical No:4

Image Enhancement in Frequency Domain.

4.(a) Aim: To employ DFT and IDFT for transitioning image data between spatial and frequency domains, supporting analysis and reconstruction.

Software used: Google Colab

Theory:

1. Discrete Fourier Transform (DFT)

DFT converts an image from its spatial domain to frequency domain representation. In the frequency domain:

- Low frequencies represent overall image structure (slow variations)
- High frequencies represent edges and fine details (rapid variations)

The DFT of an $N \times N$ image produces complex values representing magnitude and phase at different frequencies.

2. Fast Fourier Transform (FFT)

While not explicitly mentioned, OpenCV's `dft()` function uses FFT algorithms for efficient computation of DFT ($O(N \log N)$ instead of $O(N^2)$)

4A.) performing DFT and IDFT

```
[1] import cv2
import numpy as np
import matplotlib.pyplot as plt

[2] # Load the image in grayscale
image = cv2.imread('/content/lena.jpg', cv2.IMREAD_GRAYSCALE)
```

3. Frequency Shifting (fftshift)

The zero-frequency component (DC component) is shifted to the center of the spectrum for better visualization using `np.fft.fftshift()`.

```
# Compute the Discrete Fourier Transform (DFT)
dft = cv2.dft(np.float32(image), flags=cv2.DFT_COMPLEX_OUTPUT)
dft_shift = np.fft.fftshift(dft) # Shift the zero-frequency component to the center
```

4. Magnitude Spectrum

The magnitude spectrum shows how much of each frequency component exists in the image: Python code:

```
magnitude = cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1])
magnitude_spectrum = 20 * np.log(magnitude + 1)
```

- Logarithmic scaling makes low-intensity components more visible
- Adding 1 prevents $\log(0)$ which is undefined

```
# Compute the magnitude spectrum
magnitude = cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1])
magnitude_spectrum = 20 * np.log(magnitude + 1) # Add 1 to avoid log(0)
```

5. Inverse DFT (IDFT)

IDFT converts the frequency domain representation back to spatial domain:

Python code:

```
dft_ishift = np.fft.ifftshift(dft_shift) # Undo the centering idft
= cv2.idft(dft_ishift) # Compute inverse DFT
```

```
[5] # Inverse Discrete Fourier Transform (IDFT)
dft_ishift = np.fft.ifftshift(dft_shift) # Shift back
idft = cv2.idft(dft_ishift)
```

6. Image Reconstruction

The reconstructed image is obtained from the IDFT output:

Python code:

```
reconstructed_image = cv2.magnitude(idft[:, :, 0], idft[:, :, 1])
```

- `cv2.magnitude()` combines real and imaginary components
- Normalization ensures proper intensity range (0-255)

```
[6] # Reconstruct the image (using magnitude, ensuring real values)
reconstructed_image = cv2.magnitude(idft[:, :, 0], idft[:, :, 1])
```

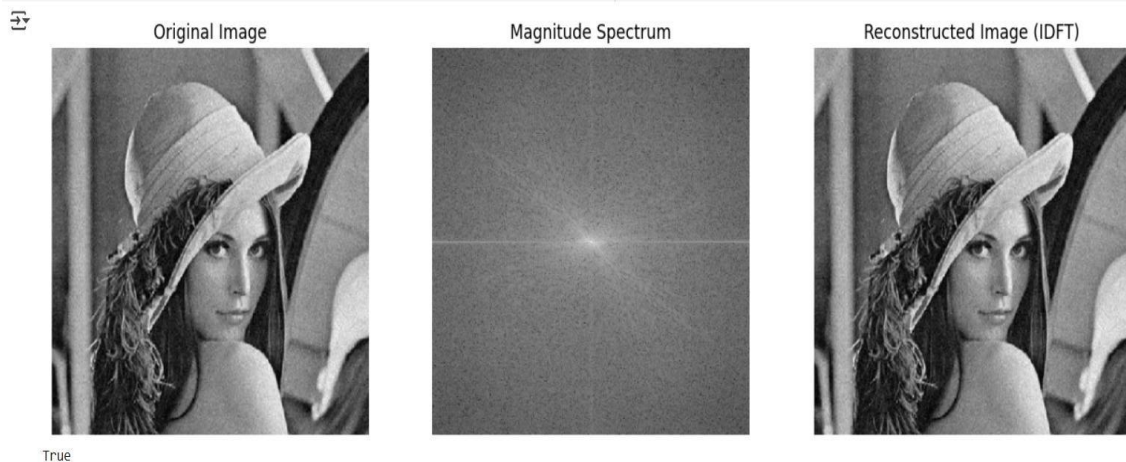
```
[7] # Normalize and convert to uint8
reconstructed_image = cv2.normalize(reconstructed_image, None, 0, 255,
cv2.NORM_MINMAX)
reconstructed_image = reconstructed_image.astype(np.uint8)
```

```
[8] # Resize the reconstructed image to match the original image's dimensions
reconstructed_image = cv2.resize(reconstructed_image, (image.shape[1],
image.shape[0]))
```

```

# Display the images using subplots with controlled aspect ratio
plt.figure(figsize=(15, 5)) # Adjusted figure size to accommodate 3 subplots
plt.subplot(1, 3, 1) # 1 row, 3 columns, first subplot
plt.imshow(image, cmap='gray', aspect='equal')
plt.title('Original Image'), plt.axis('off')
plt.subplot(1, 3, 2) # 1 row, 3 columns, second subplot
plt.imshow(magnitude_spectrum, cmap='gray', aspect='equal') # Displaying magnitude spectrum
plt.title('Magnitude Spectrum'), plt.axis('off')
plt.subplot(1, 3, 3) # 1 row, 3 columns, third subplot
plt.imshow(reconstructed_image, cmap='gray', aspect='equal')
plt.title('Reconstructed Image (IDFT)'), plt.axis('off')
plt.show()
# Save the results
cv2.imwrite('magnitude_spectrum.jpg', magnitude_spectrum)
cv2.imwrite('reconstructed_image.jpg', reconstructed_image)

```



• **Conclusion:**

The practical successfully demonstrated the application of DFT and IDFT for image processing using OpenCV and NumPy. The original grayscale image was transformed into its frequency domain, revealing its magnitude spectrum, which highlighted the distribution of low and high-frequency components. By applying the inverse DFT, the image was perfectly reconstructed, confirming the lossless nature of these transformations when no modifications are made to the frequency data. The logarithmic scaling of the magnitude spectrum improved visualization, while normalization ensured proper intensity levels in the reconstructed image. This exercise validated the reversibility of DFT-IDFT and provided a clear understanding of frequency-domain representation in images.

Practical No:4**Image Enhancement in Frequency Domain.**

4.(b) Aim: To use low pass filters to reduce high-frequency noise and achieve smoother images.

Software used: Google Colab

Theory:**1. Introduction to Low-Pass Filters (LPFs)**

A Low-Pass Filter (LPF) attenuates high-frequency components in an image while preserving low-frequency components (smooth regions). This is useful for noise reduction, blurring, and smoothing. Three common types of LPFs are:

- **Ideal LPF:** Sharp cutoff at a specified frequency (causes ringing artifacts).
- **Butterworth LPF:** Smooth transition with adjustable order (reduces ringing).
- **Gaussian LPF:** Smooth, bell-shaped transition (no ringing, optimal blurring).

2. Key Steps in the Practical:**(a) Image Loading & Frequency Domain Setup:**

- `cv2.imread()` loads the grayscale image.
- The DFT (`cv2.dft()`) converts the image to the frequency domain.
- `np.fft.fftshift()` centers the zero-frequency component.

4B) IDEAL , BUTTERWORTH AND GAUSSIAN LPFs

```
[10] import cv2
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

[11] # Load the image in grayscale
image = cv2.imread('/content/lena.jpg', cv2.IMREAD_GRAYSCALE)

[12] rows, cols = image.shape
crow, ccol = rows // 2, cols // 2 # Center of the frequency domain
```

(b) Frequency Grid & Distance Calculation

- A grid (U, V) is created to compute distances (D) from the center (DC component).
- This helps define the filter's cutoff region.

```

▶ # Frequency grid
u = np.arange(-ccol, ccol)
v = np.arange(-crow, crow)
U, V = np.meshgrid(u, v)
D = np.sqrt(U**2 + V**2)

```

(c) Filter Functions

1. Ideal LPF (ideal_lpf()):

- Binary cutoff: 1 inside cutoff radius, 0 outside.
- Causes **ringing artifacts** due to sharp transitions.

```

[14] # Filter Functions
def ideal_lpf(D, cutoff):
    H = np.zeros_like(D)
    H[D <= cutoff] = 1
    return H

```

2. Butterworth LPF (butterworth_lpf()):

- Smooth roll-off controlled by order. ○ Reduces ringing compared to Ideal LPF.

```

▶ def butterworth_lpf(D, cutoff, order):
    H = 1 / (1 + (D / cutoff)**(2 * order))
    return H

```

3. Gaussian LPF (gaussian_lpf()):

- Exponential decay, smoothest transition. ○ No ringing, best for natural blurring.

```

[16] def gaussian_lpf(D, cutoff):
    H = np.exp(-(D**2) / (2 * (cutoff**2)))
    return H

```

(d) Applying Filters (apply_filter())

1. Multiply the shifted DFT with the filter mask.
2. Inverse shift (np.fft.ifftshift()).
3. Compute **IDFT** (cv2.idft()) to return to the spatial domain.
4. Normalize (cv2.normalize()) for display.

```
[17] # Apply filters and visualize
def apply_filter(image, filter_function):
    dft = cv2.dft(np.float32(image), flags=cv2.DFT_COMPLEX_OUTPUT)
    dft_shifted = np.fft.fftshift(dft)
    filter_mask = filter_function(D)
    filtered = dft_shifted * filter_mask[:, :, np.newaxis]
    dft_ishifted = np.fft.ifftshift(filtered)
    idft = cv2.idft(dft_ishifted)
    result = cv2.magnitude(idft[:, :, 0], idft[:, :, 1])
    return filter_mask, cv2.normalize(result, None, 0, 255,
cv2.NORM_MINMAX)

✓ [18] # Filter parameters
0s  cutoff = 50
    order = 2

✓ [19] # Apply the three filters
0s  ideal_mask, ideal_result = apply_filter(image, lambda D: ideal_lpf(D,
cutoff))
    butter_mask, butter_result = apply_filter(image, lambda D:
butterworth_lpf(D, cutoff, order))
    gaussian_mask, gaussian_result = apply_filter(image, lambda D:
gaussian_lpf(D, cutoff))
```

(e) Visualization

- **Original vs. Filtered Images:** Shows blurring effects.
- **3D Transfer Functions:** Visualizes frequency response.


```

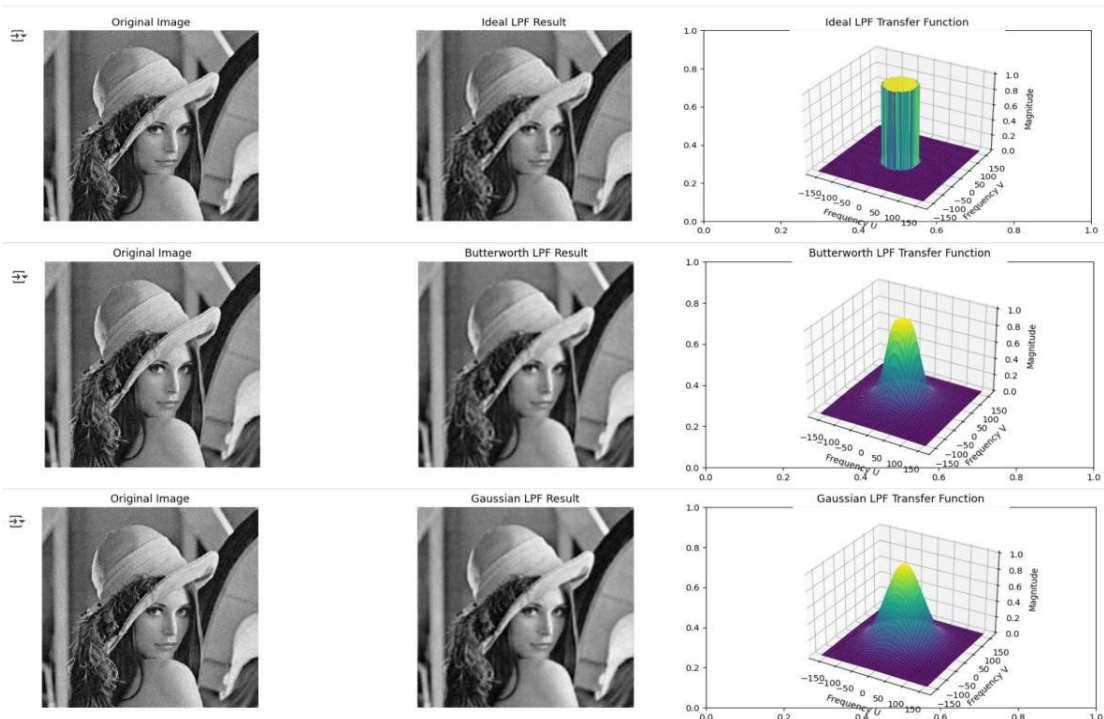
# Subplots for each filter
fig, axes = plt.subplots(3, 3, figsize=(18, 12))
# Plot for Ideal Low Pass Filter
axes[0, 0].imshow(image, cmap='gray')
axes[0, 0].set_title('Original Image')
axes[0, 0].axis('off')
axes[0, 1].imshow(np.uint8(ideal_result), cmap='gray')
axes[0, 1].set_title('Ideal LPF Result')
axes[0, 1].axis('off')
ax = fig.add_subplot(331 + 2, projection='3d')
ax.plot_surface(U, V, ideal_mask, cmap='viridis')
ax.set_title('Ideal LPF Transfer Function')
ax.set_xlabel('Frequency U')
ax.set_ylabel('Frequency V')
ax.set_zlabel('Magnitude')
# Plot for Butterworth Low Pass Filter
axes[1, 0].imshow(image, cmap='gray')
axes[1, 0].set_title('Original Image')
axes[1, 0].axis('off')
axes[1, 1].imshow(np.uint8(butter_result), cmap='gray')
axes[1, 1].set_title('Butterworth LPF Result')
axes[1, 1].axis('off')
ax = fig.add_subplot(331 + 5, projection='3d')
ax.plot_surface(U, V, butter_mask, cmap='viridis')
ax.set_title('Butterworth LPF Transfer Function')
ax.set_xlabel('Frequency U')
ax.set_ylabel('Frequency V')
ax.set_zlabel('Magnitude')

```

```

# Plot for Gaussian Low Pass Filter
axes[2, 0].imshow(image, cmap='gray')
axes[2, 0].set_title('Original Image')
axes[2, 0].axis('off')
axes[2, 1].imshow(np.uint8(gaussian_result), cmap='gray')
axes[2, 1].set_title('Gaussian LPF Result')
axes[2, 1].axis('off')
ax = fig.add_subplot(331 + 8, projection='3d')
ax.plot_surface(U, V, gaussian_mask, cmap='viridis')
ax.set_title('Gaussian LPF Transfer Function')
ax.set_xlabel('Frequency U')
ax.set_ylabel('Frequency V')
ax.set_zlabel('Magnitude')
plt.tight_layout()
plt.show()

```



- **Conclusion:**

This practical demonstrated the implementation and comparison of three lowpass filters (Ideal, Butterworth, and Gaussian) for image smoothing. The Ideal LPF produced sharp blurring but introduced ringing artifacts, while the Butterworth LPF offered smoother transitions with adjustable steepness. The Gaussian LPF delivered the most natural blur without artifacts. The 3D transfer function visualizations clearly showed how each filter attenuates high frequencies differently. Key findings confirm that Gaussian is best for artifactfree smoothing, Butterworth provides a balanced approach, and Ideal is simplest but least practical due to ringing. The results highlight the importance of selecting the right filter based on application needs.

Practical No:4**Image Enhancement in Frequency Domain.**

4.(c) Aim: To use low pass filters to reduce high-frequency noise and achieve smoother images.

Software used: Google Colab

Theory:**1. Introduction to High-Pass Filters (HPFs)**

A High-Pass Filter (HPF) attenuates low-frequency components (smooth regions) while preserving high-frequency components (edges and fine details).

This is useful for edge detection, sharpening, and texture analysis. Three common types of HPFs are:

- **Ideal HPF:** Sharp cutoff at a specified frequency (causes ringing artifacts).
- **Butterworth HPF:** Smooth transition with adjustable order (reduces ringing).
- **Gaussian HPF:** Smooth, bell-shaped transition (no ringing, optimal edge enhancement)

4C) IDEAL , BUTTERWORTH AND GAUSSIAN HPFs

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
```

2. Key Steps in the Practical:**(a) Filter Functions****1. Ideal HPF (ideal_highpass_filter()):**

- Binary cutoff: 0 inside cutoff radius, 1 outside.
- Causes ringing artifacts due to sharp transitions.

```
[22] def ideal_highpass_filter(shape, cutoff_frequency):
      """Creates an ideal high-pass filter."""
      rows, cols = shape
      crow, ccol = rows // 2, cols // 2
      mask = np.ones((rows, cols), np.uint8)
      cv2.circle(mask, (ccol, crow), cutoff_frequency, 0, -1)
      return mask
```

2. Butterworth HPF (butterworth_highpass_filter()):

- Smooth roll-off controlled by order.
- Reduces ringing compared to Ideal HPF.

```
[23] def butterworth_highpass_filter(shape, cutoff_frequency, order=2):
    """Creates a Butterworth high-pass filter."""
    rows, cols = shape
    crow, ccol = rows // 2, cols // 2
    mask = np.zeros((rows, cols), np.float32)
    for i in range(rows):
        for j in range(cols):
            dist = np.sqrt((i - crow)**2 + (j - ccol)**2)
            mask[i, j] = 1 / (1 + (cutoff_frequency / dist)**(2 * order))
    return mask
```

3. Gaussian HPF (gaussian_highpass_filter()):

- Exponential decay, smoothest transition.
- No ringing, best for natural edge enhancement.

```
[24] def gaussian_highpass_filter(shape, cutoff_frequency):
    """Creates a Gaussian high-pass filter."""
    rows, cols = shape
    crow, ccol = rows // 2, cols // 2
    mask = np.zeros((rows, cols), np.float32)
    for i in range(rows):
        for j in range(cols):
            dist = np.sqrt((i - crow)**2 + (j - ccol)**2)
            mask[i, j] = 1 - np.exp(-(dist**2) / (2 *
(cutoff_frequency**2)))
    return mask
```

(b) Image Loading & Frequency Domain Setup

- cv2.imread() loads the grayscale image.
- The DFT (cv2.dft()) converts the image to the frequency domain.
- np.fft.fftshift() centers the zero-frequency component.

```
[25] # Load the image in grayscale
image = cv2.imread('/content/lena.jpg', cv2.IMREAD_GRAYSCALE)
```

```
[26] # Compute the DFT
dft = cv2.dft(np.float32(image), flags=cv2.DFT_COMPLEX_OUTPUT)
dft_shift = np.fft.fftshift(dft)
```

(c).Magnitude Spectrum

- The magnitude spectrum shows how much of each frequency component exists in the image: • Python code:

```
magnitude = cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1])
```

```
magnitude_spectrum = 20 * np.log(magnitude + 1)
```

- Logarithmic scaling makes low-intensity components more visible

Adding 1 prevents log(0) which is undefined

```
[27] # Compute the magnitude spectrum
magnitude = cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1])
magnitude_spectrum = 20 * np.log(magnitude + 1) # Add 1 to avoid log(0)
```

(d) Applying Filters

1. Multiply the shifted DFT with the filter mask.
2. Inverse shift (`np.fft.ifftshift()`).
3. Compute IDFT (`cv2.idft()`) to return to the spatial domain.
4. Normalize (`cv2.normalize()`) for display.

```

✓ [28] # Create the filters
Os  cutoff_frequency = 30 # Adjust as needed
    ideal_mask = ideal_highpass_filter(image.shape, cutoff_frequency)
    butterworth_mask = butterworth_highpass_filter(image.shape,
    cutoff_frequency)
    gaussian_mask = gaussian_highpass_filter(image.shape, cutoff_frequency)

    <ipython-input-23-2f74487627b4>:9: RuntimeWarning: divide by zero encountered in scalar divide
    mask[i, j] = 1 / (1 + (cutoff_frequency / dist)**(2 * order))

✓ [29] # Apply the filters
Os  filtered_ideal = dft_shift * ideal_mask[:, :, np.newaxis]
    filtered_butterworth = dft_shift * butterworth_mask[:, :, np.newaxis]
    filtered_gaussian = dft_shift * gaussian_mask[:, :, np.newaxis]

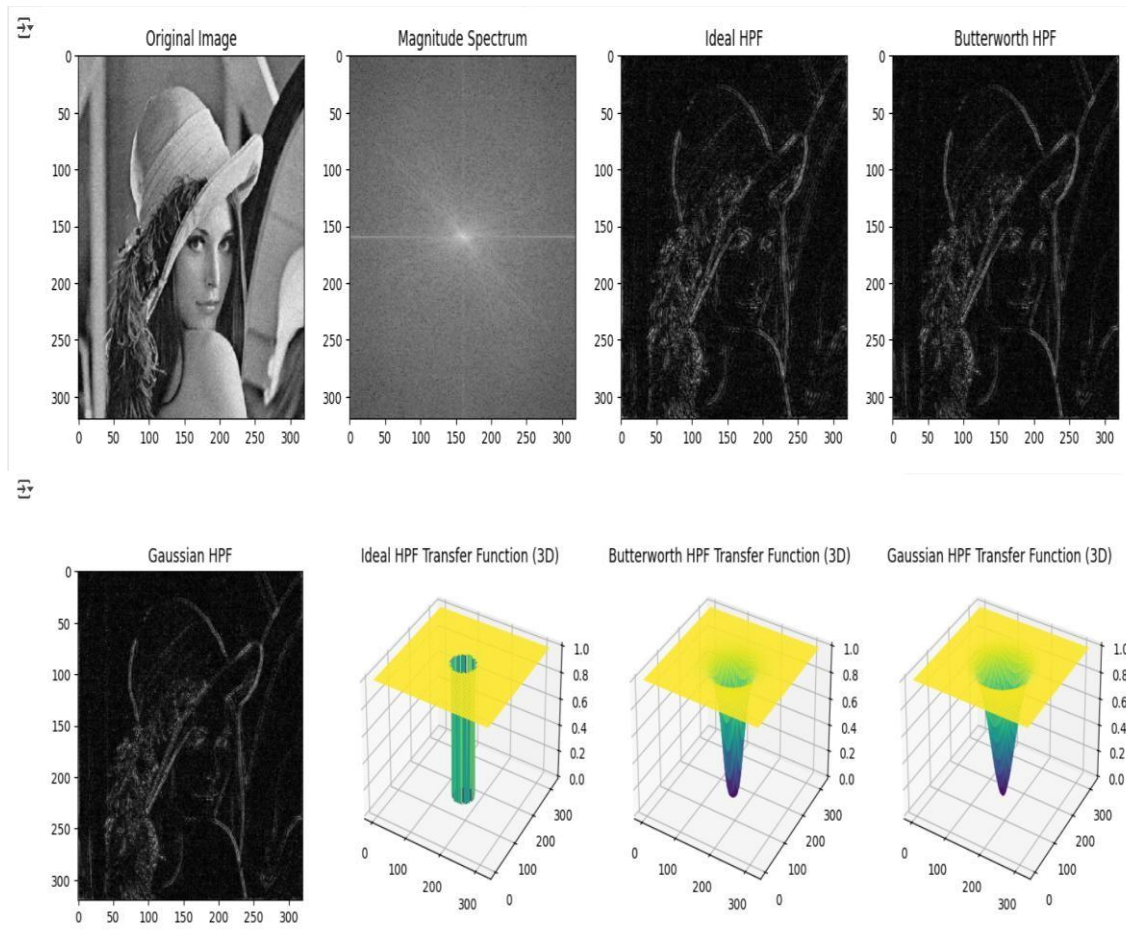
✓ [30] # Compute the IDFT
Os  ideal_ishift = np.fft.ifftshift(filtered_ideal)
    butterworth_ishift = np.fft.ifftshift(filtered_butterworth)
    gaussian_ishift = np.fft.ifftshift(filtered_gaussian)
    ideal_back = cv2.idft(ideal_ishift)
    butterworth_back = cv2.idft(butterworth_ishift)
    gaussian_back = cv2.idft(gaussian_ishift)

[31] # Reconstruct the images (using magnitude)
    ideal_image = cv2.magnitude(ideal_back[:, :, 0], ideal_back[:, :, 1])
    butterworth_image = cv2.magnitude(butterworth_back[:, :, 0],
    butterworth_back[:, :, 1])
    gaussian_image = cv2.magnitude(gaussian_back[:, :, 0], gaussian_back[:, :,
    1])

[32] # Normalize for display
    ideal_image = cv2.normalize(ideal_image, None, 0, 255,
    cv2.NORM_MINMAX).astype(np.uint8)
    butterworth_image = cv2.normalize(butterworth_image, None, 0, 255,
    cv2.NORM_MINMAX).astype(np.uint8)
    gaussian_image = cv2.normalize(gaussian_image, None, 0, 255,
    cv2.NORM_MINMAX).astype(np.uint8)

▶ # Display the results
    plt.figure(figsize=(18, 12)) # Adjusted figure size for more subplots
    # Original Image and Magnitude Spectrum
    plt.subplot(2, 4, 1), plt.imshow(image, cmap='gray'), plt.title('Original Image')
    plt.subplot(2, 4, 2), plt.imshow(magnitude_spectrum, cmap='gray'),
    plt.title('Magnitude Spectrum')
    # Filtered Images
    plt.subplot(2, 4, 3), plt.imshow(ideal_image, cmap='gray'),
    plt.title('Ideal HPF')
    plt.subplot(2, 4, 4), plt.imshow(butterworth_image, cmap='gray'),
    plt.title('Butterworth HPF')
    plt.subplot(2, 4, 5), plt.imshow(gaussian_image, cmap='gray'),
    plt.title('Gaussian HPF')
    # Perspective Plots of Transfer Functions
    X, Y = np.meshgrid(np.arange(ideal_mask.shape[1]),
    np.arange(ideal_mask.shape[0]))
    ax1 = plt.subplot(2, 4, 6, projection='3d')
    ax1.plot_surface(X, Y, ideal_mask, cmap='viridis')
    ax1.set_title('Ideal HPF Transfer Function (3D)')
    ax2 = plt.subplot(2, 4, 7, projection='3d')
    ax2.plot_surface(X, Y, butterworth_mask, cmap='viridis')
    ax2.set_title('Butterworth HPF Transfer Function (3D)')
    ax3 = plt.subplot(2, 4, 8, projection='3d')
    ax3.plot_surface(X, Y, gaussian_mask, cmap='viridis')
    ax3.set_title('Gaussian HPF Transfer Function (3D)')
    plt.show()

```

• **Conclusion:**

This practical successfully demonstrated the implementation and comparison of three high-pass filters (Ideal, Butterworth, and Gaussian) for edge enhancement. The Ideal HPF produced sharp edges but introduced ringing artifacts, while the Butterworth HPF offered smoother transitions with adjustable steepness. The Gaussian HPF delivered the cleanest edge enhancement without artifacts. The 3D transfer function visualizations clearly showed each filter's frequency response characteristics. Key findings confirm that Gaussian is optimal for artifact-free edge enhancement, Butterworth provides a good balance, and Ideal is simplest but suffers from ringing. The results highlight how different HPFs can be applied based on specific image processing needs, with Gaussian being most suitable for natural-looking edge enhancement.

Practical No:5

Applications of Image Denoising Techniques.

Aim: To enhance the quality of degraded images by restoring clarity using inverse filtering and minimizing noise and blur effects through Wiener filtering techniques.

Software used: Google Colab

Theory:

A. Motion Blur Simulation

Purpose: Artificially creates directional blur to simulate camera/object motion.

Implementation:

Python code:

```
def motion_blur(image, kernel_size, angle):
    kernel = np.zeros((kernel_size, kernel_size))
    kernel[center_row, :] = np.ones(kernel_size) # Create a line kernel
    kernel = cv2.warpAffine(kernel, rotation_matrix, ...) # Rotate to specified angle
    kernel = kernel / np.sum(kernel) # Normalize
    return convolve2d(image, kernel, mode='same') # Apply blur
```

Parameters: ○ kernel_size: Controls blur length.

○ angle: Direction of motion (degrees).

```
!pip install numpy scipy matplotlib opencv-python
import numpy as np
import cv2
import matplotlib.pyplot as plt
from scipy.signal import convolve2d
```

```
[2] def motion_blur(image, kernel_size, angle):
    """Applies motion blur to an image.
    Args:
        image: The input image.
        kernel_size: The size of the motion blur kernel.
        angle: The angle of the motion blur in degrees.
    Returns:
        The blurred image.
    """
    kernel = np.zeros((kernel_size, kernel_size))
    kernel[int((kernel_size - 1) / 2), :] = np.ones(kernel_size)
    kernel = cv2.warpAffine(kernel, cv2.getRotationMatrix2D((kernel_size / 2 - 0.5, kernel_size / 2 - 0.5), angle, 1.0), (kernel_size, kernel_size))
    kernel = kernel / np.sum(kernel)
    blurred = convolve2d(image, kernel, mode='same', boundary='wrap')
    return blurred
```

B. Inverse Filtering

- **Purpose:** Naive deblurring by dividing the blurred image's spectrum by the kernel's spectrum.
- **Formula:**

$$|\hat{F}(u,v) = G(u,v) / H(u,v)|$$
 where:
 - $G(u,v)$: Blurred image's DFT
 - $H(u,v)$: Blur kernel's DFT
- **Implementation:**

Python code:

```
deblurred_fft = blurred_fft / (kernel_fft + eps) # eps prevents division by zero
```

- **Limitation:** Amplifies noise if $H(u,v)$ has near-zero values

```
def inverse_filter(blurred, kernel, eps):
    """Applies inverse filtering to deblur an image.
    Args:
        blurred: The blurred image.
        kernel: The blur kernel.
        eps: A small value to avoid division by zero.
    Returns:
        The deblurred image.
    """
    blurred_fft = np.fft.fft2(blurred)
    kernel_fft = np.fft.fft2(kernel, s=blurred.shape)
    deblurred_fft = blurred_fft / (kernel_fft + eps) # Add eps to avoid division by zero
    deblurred = np.fft.ifft2(deblurred_fft).real
    return deblurred
```

C. Wiener Filtering

- **Purpose:** Robust deblurring that accounts for noise.
- **Formula:**

$$|\hat{F}(u,v) = [H^*(u,v) / (|H(u,v)|^2 + K)] \cdot G(u,v)|$$
 where:
 - K : Noise-to-signal ratio (regularization term).
- **Implementation:**

Python Code:

```
deblurred_fft = (kernel_fft_conj / (|kernel_fft|^2 + k)) * blurred_fft
```

- **Advantage:** Suppresses noise amplification compared to Inverse Filtering.

```
[4] def wiener_filter(blurred, kernel, k):
    """Applies Wiener filtering to deblur an image.
    Args:
        blurred: The blurred image.
        kernel: The blur kernel.
        k: The noise-to-signal ratio.
    Returns:
        The deblurred image.
    """
    blurred_fft = np.fft.fft2(blurred)
    kernel_fft = np.fft.fft2(kernel, s=blurred.shape)
    kernel_fft_conj = np.conj(kernel_fft)
    deblurred_fft = (kernel_fft_conj / (np.abs(kernel_fft)**2 + k)) * blurred_fft
    deblurred = np.fft.ifft2(deblurred_fft).real
    return deblurred
```

```
0s [5] # Load image and apply motion blur
img = cv2.imread('/content/lena.jpg', cv2.IMREAD_GRAYSCALE)
kernel_size = 21
angle = 11
blurred = motion_blur(img, kernel_size, angle)
```

```
1s # Create blur kernel
kernel = np.zeros((kernel_size, kernel_size))
kernel[int((kernel_size - 1) / 2), :] = np.ones(kernel_size)
kernel = cv2.warpAffine(kernel, cv2.getRotationMatrix2D((kernel_size / 2 - 0.5, kernel_size / 2 - 0.5), angle, 1.0), (kernel_size, kernel_size))
kernel = kernel / np.sum(kernel)
# Deblur using inverse filtering
deblurred_inverse = inverse_filter(blurred, kernel, 0.01)
# Deblur using Wiener filtering
deblurred_wiener = wiener_filter(blurred, kernel, 0.01)
# Display results
plt.figure(figsize=(12, 6))
plt.subplot(131), plt.imshow(img, cmap='gray'), plt.title('Original Image')
plt.subplot(132), plt.imshow(blurred, cmap='gray'), plt.title('Blurred Image')
plt.subplot(133), plt.imshow(deblurred_inverse, cmap='gray'),
plt.title('Inverse Filtered')
plt.show()
plt.figure(figsize=(12, 6))
plt.subplot(131), plt.imshow(img, cmap='gray'), plt.title('Original Image')
plt.subplot(132), plt.imshow(blurred, cmap='gray'), plt.title('Blurred Image')
plt.subplot(133), plt.imshow(deblurred_wiener, cmap='gray'),
plt.title('Wiener Filtered')
plt.show()
```



- **Conclusion:**

This practical demonstrates that motion blur can be artificially generated and reversed using frequency-domain techniques. While inverse filtering provides a basic restoration method, Wiener filtering is more effective in real-world scenarios due to its noise-handling capabilities. The choice between methods depends on the presence of noise and the accuracy of the blur kernel estimation.

Practical No:6

Illustrations of various Segmentation Techniques.

Aim: To demonstrate segmentation techniques by implementing edge detection methods such as Sobel, Prewitt, Canny, and Laplacian operators to define image boundaries.

Software used: Google Colab

Theory:

1. Introduction to Edge Detection:

Edge detection identifies boundaries and sharp transitions in images by detecting intensity changes. This practical demonstrates four key edge detection operators: Sobel, Prewitt, Canny, and Laplacian, each with distinct characteristics and applications.

. Image Loading & Frequency Domain Setup

- cv2.imread() loads the grayscale image.

```
[1] import cv2
import numpy as np
import matplotlib.pyplot as plt

[2] # Read and convert the image to grayscale
image = cv2.imread('/content/lena.jpg', cv2.IMREAD_GRAYSCALE)
```

2. Key Edge Detection Techniques

A. Sobel Operator

- **Purpose:** Detects horizontal and vertical edges separately.
- **Method:**
 - Uses 3×3 convolution kernels to approximate gradients.
 - Sobel X (horizontal edges): Kernel emphasizes vertical changes.
 - Sobel Y (vertical edges): Kernel emphasizes horizontal changes.
- **Advantages:** Smooths noise while computing gradients.
- **Function:**

Python code:

```
cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3) # Horizontal (X)
cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3) # Vertical (Y)
```

```
[3] # Sobel operator (Horizontal and Vertical)
sobel_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3) # Horizontal
sobel_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3) # Vertical
```

B. Prewitt Operator

- **Purpose:** Simpler gradient-based edge detection.
- **Method:**
 - Uses fixed 3×3 kernels for horizontal (Prewitt X) and vertical (Prewitt Y) edges.
 - Less noise smoothing compared to Sobel.
- **Function:**

Python code: `cv2.filter2D(image, -1, kernel_x)`

Horizontal

```
[4] # Prewitt operator (Horizontal and Vertical)
kernel_x = np.array([[ -1,  0,  1], [ -1,  0,  1], [ -1,  0,  1]])
kernel_y = np.array([[ 1,  1,  1], [ 0,  0,  0], [ -1, -1, -1]])
prewitt_x = cv2.filter2D(image, -1, kernel_x) # Horizontal
prewitt_y = cv2.filter2D(image, -1, kernel_y) # Vertical
```

C. Canny Edge Detector

- **Purpose:** Multi-stage algorithm for optimal edge detection.
- **Steps:**
 1. **Noise Reduction:** Gaussian blur.
 2. **Gradient Calculation:** Uses Sobel operators.
 3. **Non-Maximum Suppression:** Thins edges.
 4. **Hysteresis Thresholding:** Discards weak edges (100 and 200 as thresholds).
- **Advantages:** Balances noise robustness and edge precision.
- **Function:**

Python code:

`cv2.Canny(image, 100, 200)`

```
[5] # Canny operator
canny_edges = cv2.Canny(image, 100, 200)
```

D. Laplacian Operator

- **Purpose:** Detects edges by analyzing second-order derivatives.
- **Method:**
 - Highlights regions of rapid intensity change (zero-crossings).
 - Sensitive to noise; often paired with Gaussian blur (LoG).
- **Function:**

Python code: `cv2.Laplacian(image,`
`cv2.CV_64F)`

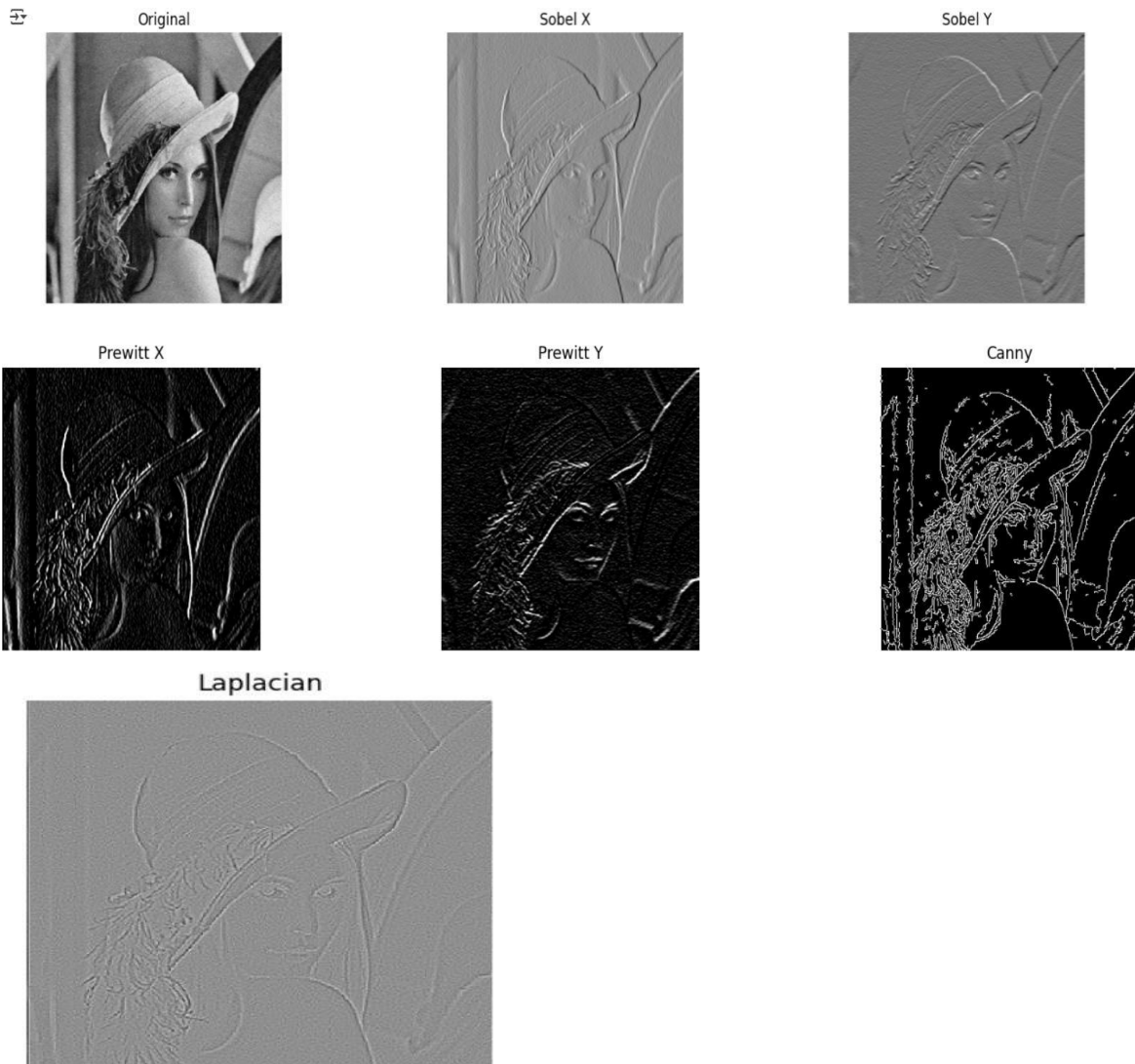
```
[6] # Laplacian operator
    laplacian = cv2.Laplacian(image, cv2.CV_64F)
```

- Display Results:

```
[7] # Display results
    titles = ['Original', 'Sobel x', 'Sobel Y', 'Prewitt x', 'Prewitt Y',
             'Canny', 'Laplacian']
    images = [image, sobel_x, sobel_y, prewitt_x, prewitt_y, canny_edges,
             laplacian]
```

```
plt.figure(figsize=(15, 10))
for i in range(len(images)):
    plt.subplot(3, 3, i + 1)
    plt.imshow(images[i], cmap='gray')
    plt.title(titles[i])
    plt.axis('off')

plt.tight_layout()
plt.show()
```



- **Conclusion:**

This practical compared four edge detection techniques—Sobel, Prewitt, Canny, and Laplacian—demonstrating their unique strengths. The Sobel operator provided smooth directional edges, while Prewitt offered a simpler but noisier alternative. Canny delivered the cleanest, most precise edges through its multi-stage process, and Laplacian highlighted fine details but amplified noise. Key takeaways show Canny as the most effective for robust edge detection, while Sobel and Prewitt serve well for basic gradient analysis. The results underscore that choosing the right operator depends on balancing noise tolerance, edge precision, and computational needs.

Practical No:7

Applications of Morphological operations on Images.

Aim: To use morphological operations, including erosion, dilation, opening, and closing, for refining image structures and extracting key features.

Software used: Google Colab

Theory:

1. Introduction to Morphological Operations:

Morphological operations are fundamental techniques in image processing used to analyze and manipulate the structure of objects within binary or grayscale images. These operations are particularly useful for tasks like noise removal, edge detection, and shape analysis. The core operations include erosion, dilation, opening, and closing, each serving distinct purposes in modifying image features.

. Image Loading & Frequency Domain Setup

- cv2.imread() loads the grayscale image.

```
[1] import cv2
import numpy as np
import matplotlib.pyplot as plt

[2] # Load the image
image = cv2.imread('/content/blobs.png', cv2.IMREAD_GRAYSCALE)
```

2. Key Morphological Operations

A. Erosion

- **Purpose:** Shrinks or thins objects in an image by removing pixels from their boundaries.
- **Method:** ○ A structuring element (kernel) slides over the image. ○ If all pixels under the kernel match, the center pixel is retained; otherwise, it is removed.
- **Effect:** ○ Removes small noise and disconnects weakly linked objects.

- **Function:**

Python code: cv2.erode(image,
kernel, iterations=1)

```
# Create a structuring element
kernel = np.ones((5,5), np.uint8)
# Perform erosion
erosion = cv2.erode(image, kernel, iterations = 1)
```

B. Dilation

- **Purpose:** Expands or thickens objects in an image by adding pixels to their boundaries.
- **Method:** ○ The structuring element slides over the image. ○ If any pixel under the kernel matches, the center pixel is set to 1 (or retained).
- **Effect:** ○ Fills small holes and connects broken parts of objects.
- **Function:**

Python code: cv2.dilate(image,
kernel, iterations=1)

```
# Perform dilation
dilation = cv2.dilate(image, kernel, iterations = 1)
```

C. Opening

- **Purpose:** Combines erosion followed by dilation to remove noise while preserving object shape.
- **Method:** ○ Erosion removes small noise, and dilation restores the remaining objects.
- **Effect:** ○ Smooths object contours and eliminates thin protrusions.
- **Function:**

Python code: cv2.morphologyEx(image,
cv2.MORPH_OPEN, kernel)

```
# Perform opening
opening = cv2.morphologyEx(image, cv2.MORPH_OPEN, kernel)
```

D. Closing

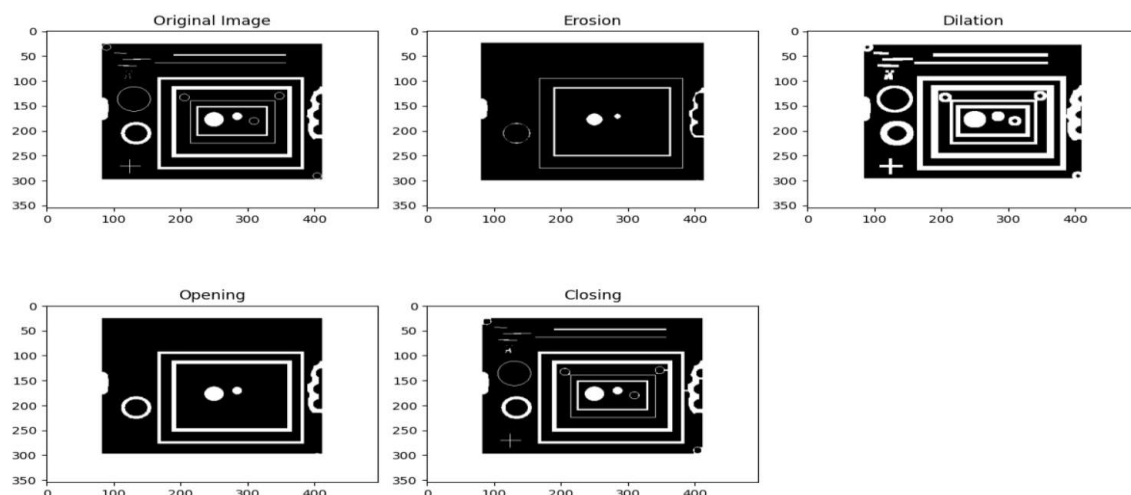
- **Purpose:** Combines dilation followed by erosion to fill holes while preserving object shape.
- **Method:** ○ Dilation fills gaps, and erosion refines the boundaries. •
- **Effect:** ○ Closes small holes and joins narrow breaks in objects.
- **Function:**

Python code: `cv2.morphologyEx(image, cv2.MORPH_CLOSE, kernel)`

```
# Perform closing
closing = cv2.morphologyEx(image, cv2.MORPH_CLOSE, kernel)
```

- Display the results in subplots:

```
plt.figure(figsize=(12, 8))
plt.subplot(2, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.subplot(2, 3, 2)
plt.imshow(erosion, cmap='gray')
plt.title('Erosion')
plt.subplot(2, 3, 3)
plt.imshow(dilation, cmap='gray')
plt.title('Dilation')
plt.subplot(2, 3, 4)
plt.imshow(opening, cmap='gray')
plt.title('Opening')
plt.subplot(2, 3, 5)
plt.imshow(closing, cmap='gray')
plt.title('Closing')
plt.tight_layout()
plt.show()
```



- **Conclusion:**

This practical demonstrated four key morphological operations - erosion, dilation, opening, and closing - using a 5×5 kernel on a grayscale image. Erosion effectively shrank objects and removed small noise, while dilation expanded them and filled gaps. Opening (erosion followed by dilation) cleaned up small artifacts without distorting main features, and closing (dilation followed by erosion) connected nearby objects while filling holes. The results clearly showed how these operations can modify image structures, with each technique serving distinct purposes in image preprocessing. The practical highlighted that morphological operations are powerful tools for noise removal, gap filling, and object separation, with the choice depending on whether the goal is to remove, preserve, or enhance specific image features.